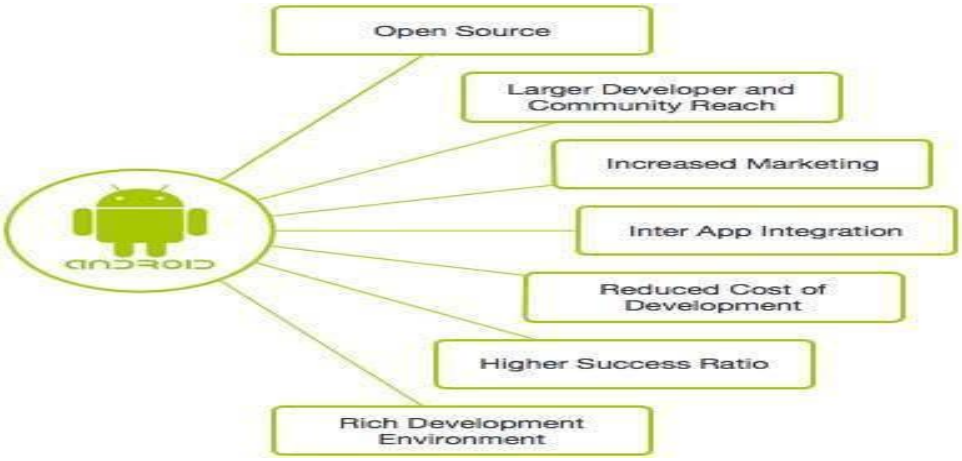
HANDOUT

UNIT I - ANDROID OVERVIEW

Introduction – Android SDK features – OHA – Development framework – Getting started –developing for android, mobile devices - ADT – Creating an applications and activities -Application manifest – Android Application Life Cycle – Understanding application priority -Externalizing resources – Android application class – Android Activities.

INTRODUCTION

- Android is an open source and Linux-based **Operating System** for mobile devices such as smartphones and tablet computers.
- Android was developed by the *Open Handset Alliance*, led by Google, and other companies.
- Android offers a unified approach to application development for mobile devices which means developers need only develop for Android, and their applications should be able to run on different devices powered by Android.



Features of Android:

Beautiful UI

Android OS basic screen provides a beautiful and intuitive user interface.

Connectivity

GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth, Wi-Fi, LTE, NFC and WiMAX.

Storage

SQLite, a lightweight relational database, is used for data storage purposes.

Media support

H.263, H.264, MPEG-4 SP, AMR, AMR-WB, AAC, HE-AAC, AAC 5.1, MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP.

Messaging - SMS and MMS

Web browser

Based on the open-source WebKit layout engine, coupled with Chrome's V8 JavaScript engine supporting HTML5 and CSS3.

Android applications:

- Entertainment
- Tools
- Communication
- Productivity
- Personalization
- Music and Audio
- Social
- Media and Video
- Travel and Local

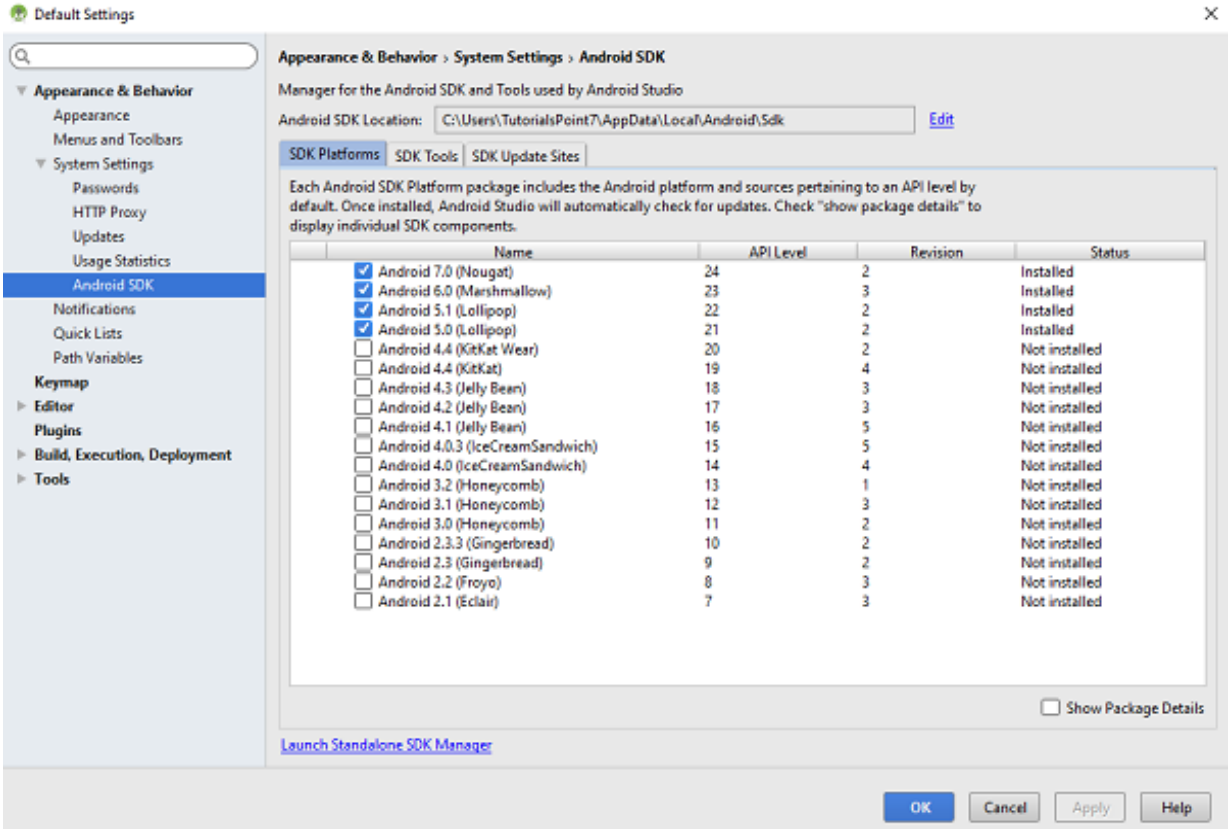
ANDROID SDK FEATURES

- Android SDK stands for Android Software Development Kit which is developed by Google for Android Platform. With the help of Android SDK, we can create android Apps easily.
- Android SDK is a collection of libraries and Software Development tools that are essential for Developing Android Applications.
- Android SDK consists of some tools which are very essential for the development of Android Application.
- These tools provide a smooth flow of the development process from developing and debugging. Android SDK is compatible with all operating systems such as Windows, Linux, macOS, etc.
- To download and install latest android APIs and development tools from the internet, android provide us with android SDK manager.
- Android SDK Manager separates the APIs, tools and different platforms into different packages which you can download.

- Android SDK manager comes with the Android SDK bundle. You can't download it separately.

Running Android SDK Manager

- Once downloaded, you can launch Android SDK Manager in one of the following ways −
- Click tools->Android-> SDK Manager option in Eclipse.
- Double Click on the SDK Manager.exe file in the Android SDK folder



Components of Android SDK:

1. Android SDK Tools

- Android SDK tool is an important component of Android SDK. It consists of a complete set of development and debugging tools. Below are the SDK developer tools:
- Android SDK Build tool.
- Android Emulator.
- Android SDK Platform-tools.
- Android SDK Tools.

2. Android SDK Build-Tools

Android SDK build tools are used for building actual binaries of Android App. The main

functions of Android SDK Build tools are built, debug, run and test Android applications.

3. Android Emulator

- An Android Emulator is a device that simulates an Android device on your system. Suppose we want to run our android application that we code.
- In Android Emulator the virtual android device is shown on our system on which we run the Android application that we code.

4. Android SDK Platform-tools

❖ Android SDK Platform-tools is helpful when we are working on Project and they will show the error messages at the same time. It is specifically used for testing. It includes:

✓ Android Debug Bridge (ADB), is a command-line tool that helps to communicate with the device. It allows us to perform an action such as Installing App and Debugging App etc.

✓ Fastboot allows you to flash a device with a new system image.

✓ Systrace tools help to collect and inspect timing information. It is very crucial for App Debugging.

OPEN HANDSET ALLIANCE (OHA)

- The Open Handset Alliance (OHA) is a consortium of 84 firms to develop open standards for mobile devices.
- Established on 5th November 2007, as a consortium of 84 companies, the Open Handset Alliance (OHA) includes Google, Samsung, AKM, Synaptics, KDDI, Garmin, Teleca, eBay, Intel, etc.
- The Open Handset Alliance (OHA) is led by Google. The main objective of Android is to advance open standards, provide services, and deploy handsets.

DEVELOPMENT FRAMEWORK

- In general, a framework is a real or conceptual structure intended to serve as a support or guide for the building of something that expands the structure into something useful.
- An Android application framework is a software toolkit that enables app developers to piece together a finished product that meets the requirements of its proprietor.
- A framework provides the bones of an application, to be fleshed out with graphics, animation, special features and functionality.
- The Application Framework layer provides many higher-level services to applications in the form of Java classes.

- Android architecture or Android software stack is categorized into five parts:
- Linux kernel
- Native libraries (middleware)
- Android Runtime
- Application Framework
- Application Layer

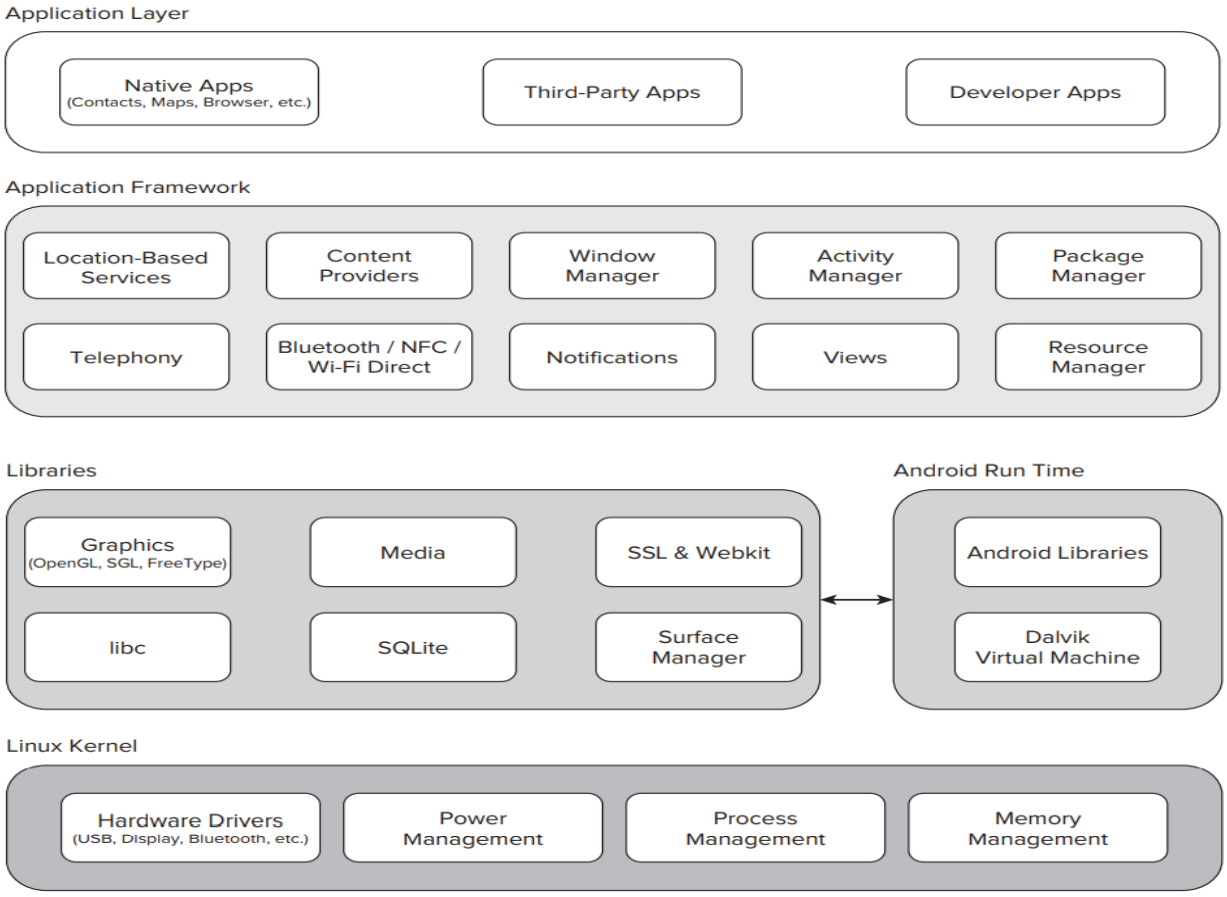


FIGURE 1-1

1) Linux kernel

- It is the heart of android architecture that exists at the root of android architecture.
- **Linux kernel** is responsible for device drivers, power management, memory management, device management and resource access.

2) Native Libraries

- Running on the top of linux kernel, Native libraries such as WebKit, OpenGL, FreeType, SQLite, Media, C runtime library (libc) etc.
- The WebKit library is responsible for browser support and Internet security;
- SQLite is for database, FreeType for font support,
- Media for playing and recording audio and video formats

Some key core Android libraries available to the Android developer is as follows −

- **android.app** − Provides access to the application model and is the cornerstone of all Android applications.
- **android.content** − Facilitates content access, publishing and messaging between applications and application components.
- **android.database** − Used to access data published by content providers and includes SQLite database management classes.
- **android.opengl** − A Java interface to the OpenGL ES 3D graphics rendering API.
- **android.os** − Provides applications with access to standard operating system services including messages, system services and inter-process communication.
- **android.text** − Used to render and manipulate text on a device display.
- **android.view** − The fundamental building blocks of application user interfaces.
- **android.widget** − A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
- **android.webkit** − A set of classes intended to allow web-browsing capabilities to be built into applications.

3) Android Runtime

- In android runtime, there are core libraries and DVM (Dalvik Virtual Machine) which is responsible to run android application.
- One of the key elements of Android is the Dalvik VM, Android uses its own custom VM designed to ensure that multiple instances run efficiently on a single device.
- DVM is like JVM but it is optimized for mobile devices.
- The Dalvik VM makes use of Linux core features like memory management and multi-threading, which is intrinsic in the Java language.
- It consumes less memory and provides fast performance.

4) Android Framework

- On the top of Native libraries and android runtime, there is android framework.
- Android framework includes Android API's such as UI (User Interface), telephony, resources, locations, Content Providers (data) and package managers.
- It provides a lot of classes and interfaces for android application development.

GETTING STARTED –DEVELOPING FOR ANDROID, MOBILE DEVICES

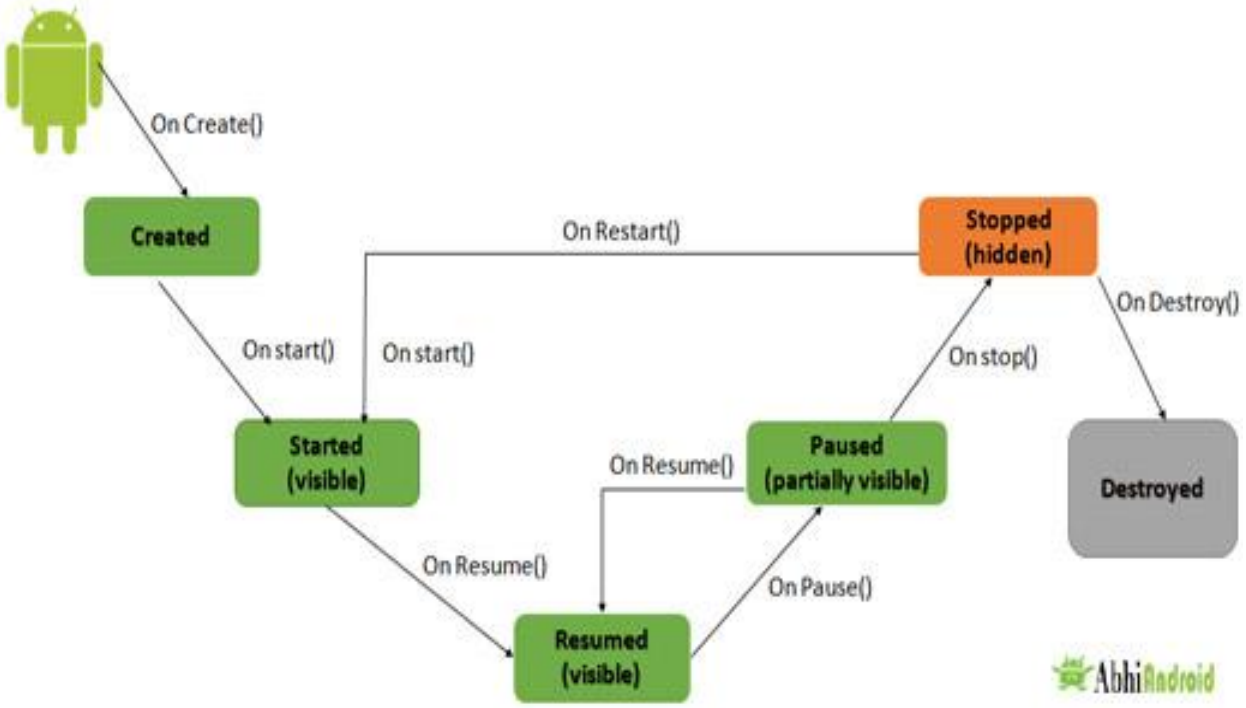
ANDROID ACTIVITY LIFECYCLE

- Activity is one of the building blocks of Android OS.
- Activity is a screen that user interact with.
- Every Activity in android has lifecycle like created, started, resumed, paused, stopped or destroyed.
- Activity is a class pre-written in Java Programming. An activity is the single screen in android. It is like window or frame of Java.

Android Activity Lifecycle methods

The 7 lifecycle methods of Activity describe how activity will behave at different states.

- onCreate()
- onStart()
- onResume()
- onPause()
- onStop()
- onRestart()
- onDestroy
- onCreate() – Called when the activity is first created.
- onStart() – Called just after its creation or by restart method after onStop(). Here Activity start becoming visible to user.
- onResume() – Called when Activity is visible to user and user can interact with it
- onPause() – Called when Activity content is not visible because user resume previous activity.
- onStop() – Called when activity is not visible to user because some other activity takes place of it
- onRestart() – Called when user comes on screen or resume the activity which was stopped
- onDestroy – Called when Activity is not in background



Method	Description	Next
onCreate()	Called when the activity is first created.	onStart()
onRestart()	Called after your activity has been stopped, just prior to it being started again. Always followed by onStart()	onStart()
onStart()	Called when the activity is becoming visible to the user. Followed by onResume() if the activity comes to the foreground, or onStop() if it becomes hidden.	onResume()or onStop()
onResume()	Called when the activity will start interacting with the user. Always followed by onPause().	onPause()
onPause()	Called when the system is about to start resuming a previous activity.	onResume() or onStop()
onStop()	Called when the activity is no longer visible to the user. Followed by either onRestart() if this activity is coming back to interact with the user, or onDestroy() if this activity is going away.	onRestart() or onDestroy()
onDestroy()	The final call you receive before your activity is destroyed.	nothing

EXAMPLE

Following is the content of the modified main activity file src/com.example.helloworld/MainActivity.java. This file includes each of the fundamental life cycle methods.

The Log.d() method has been used to generate log messages –

```
package com.example.helloworld;
import android.os.Bundle;
import android.app.Activity;
import android.util.Log;
public class MainActivity extends Activity {
	String msg = "Android : ";
	/** Called when the activity is first created. */
	@Override
	public void onCreate(Bundle savedInstanceState) {
	super.onCreate(savedInstanceState);  setContentView(R.layout.activity_main);
		Log.d(msg, "The onCreate() event"); }
	/** Called when the activity is about to become visible. */ @Override
	protected void onStart() {
		super.onStart();
		Log.d(msg, "The onStart() event"); }
/** Called when the activity has become visible. */
	@Override
	protected void onResume() {
		super.onResume();
		Log.d(msg, "The onResume() event"); }
	/** Called when another activity is taking focus. */
	@Override
	protected void onPause() {
		super.onPause();
		Log.d(msg, "The onPause() event"); }
/** Called when the activity is no longer visible. */
	@Override
	protected void onStop() {
```

```
        super.onStop();
        Log.d(msg, "The onStop() event"); }
    /** Called just before the activity is destroyed. */
    @Override
    public void onDestroy() {
        super.onDestroy();
        Log.d(msg, "The onDestroy() event");
    } }
```

ANDROID DEVELOPMENT TOOLS

- Android Development Tools (ADT) is a plug-in for the Eclipse IDE that is designed to give you a powerful, integrated environment in which to build Android applications.
- ADT is the Android Developer Tools plug-in for Eclipse.
- ADT is about the useful code stuff: for example the libraries, packages.
- The ADT plug-in for Eclipse uses the SDK tools as part of its work.

ADT plug-in incorporates many of the tools into the Eclipse IDE, where you can access them from the DDMS perspective, including the following:

- Android Virtual Device and SDK Managers
- Android Emulator
- Dalvik Debug Monitoring Service (DDMS)
- Android Debug Bridge (ADB)
- Android Asset Packaging Tool (AAPT)
- SQLite3
- MkSDCard

Android Virtual Device and SDK Managers

- Used to create and manage AVDs and to download SDK packages, respectively.
- The AVD hosts an Emulator running on a particular build of Android,
- It specify the supported SDK version, screen resolution, amount of SD card storage available, and available hardware capabilities (such as touchscreens and GPS).

Android Emulator

- Use the Emulator to test and debug your Android applications

Dalvik Debug Monitoring Service (DDMS)

- Use the DDMS to monitor and control the Emulators on which you’re debugging your applications.

Android Debug Bridge (ADB)

- A client-server application that provides a link to virtual and physical devices. It lets you copy files, install compiled application packages (.apk), and run shell commands.

Android Asset Packaging Tool (AAPT)

- Constructs the distributable Android package fi les (.apk).

SQLite3

- A database tool that you can use to access the SQLite database fi les created and used by Android.

MkSDCard

- Creates an SD card disk image that can be used by the Emulator to simulate an external storage card.

CREATING AN APPLICATIONS AND ACTIVITIES

- Used to create and manage AVDs and to download SDK packages, respectively.
- The AVD hosts an Emulator running on a particular build of Android,
- It specify the supported SDK version, screen resolution, amount of SD card storage available, and available hardware capabilities (such as touchscreens and GPS).

ANDROIDMANIFEST.XML_FILE IN ANDROID

- The AndroidManifest.xml file contains information of your package, including components of the application such as activities, services, broadcast receivers, content providers etc.
- It is responsible to protect the application to access any protected parts by providing the permissions.

Elements of the AndroidManifest.xml file

- <manifest>
- manifest is the root element of the AndroidManifest.xml file. It has package attribute that describes the package name of the activity class.
- <application>
- application is the subelement of the manifest. It includes the namespace declaration. This element contains several subelements that declares the application component such as activity etc.
- The commonly used attributes are of this element are icon, label, theme etc.
- android:icon represents the icon for all the android application components.
- android:label works as the default label for all the application components.

- android:theme represents a common theme for all the android activities.

EXTERNALIZING RESOURCES

Android Resources Resources and Internationalization

Resources are external files (that is, non-code files) that are used by your code and compiled into your application at build time.

Android supports a number of different kinds of resource files, including XML, PNG, and JPEG files.

Resources are externalized from source code, and XML files are compiled into a binary, fast loading format for efficiency reasons. Strings, likewise, are compressed into a more efficient storage form.

Android Resources Using Resources

Android resource system keeps track of all non-code assets associated with an application.

To use the Resources class to access your application's resources; the Resources instance associated with your application can generally be found through Context.getResources().

To use a resource, you must install it correctly in the source tree and build your application.

Android – Resources Example

Try to install the ApiDemos application. Explore its resource folder. Find the source code in the folder:

c:\Android\platforms\android-1.6\samples\

How to install the App:

File -> New -> Project -> Android Project -> Next

Select "Create project form existing source"

Select the ApiDemos folder (all remaining fields will be self adjusted)

Android Resources Java Statements for Using Resources

Displaying a screen layout:

setContentView(R.layout.main);

setContentView(R.layout.screen2);

Android Resources Java Statements for Using Resources

Retrieving String Resources from: res/values/…

/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<string name="hello">Hola Mundo!, ResourceDemo1!</string>
<string name="app_name">ResourceDemo1</string>
```

```
<string name="good_bye">Hasta luego</string>
<string name="color_caption">Color:</string>
<string name="color_prompt">Seleccione un Color</string>
<string name="planet_caption">
<b>Planeta </b>Planeta <i>Planeta </i><u>Planeta: </u></string>
<string name="planet_prompt">Seleccione un Planeta</string>
</resources>
String msg =
this.getString(R.string.color_prompt);
```

Android Resources Java Statements for Using Resources

Enhancing externalized String resources from: res/values/…

/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<string name="hello">Hola Mundo!, ResourceDemo1!</string>
<string name="app_name">ResourceDemo1</string>
<string name="good_bye">Hasta luego</string>
<string name="color_caption">Color:</string>
<string name="color_prompt">Seleccione un Color</string>
<string name="planet_caption">
<b>Planeta </b>Planeta <i>Planeta </i><u>Planeta: </u></string>
<string name="planet_prompt">Seleccione un Planeta</string>
</resources>
```

Android Resources

Example1. Using Embedded Resources (drawable, string, array).

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="
<TextView
android:orientation="vertical"
android:layout_marginTop="10dip"
android:padding="10dip"
android:textSize="18px"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
```

```
>
<Spinner
<ImageView
android:drawSelectorOnTop="true"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
</ImageView>
</LinearLayout>
<EditText
/>
<?xml version="1.0" encoding="utf-8"?>
<resources>
<drawable name="red">#7f00</drawable>
<drawable name="blue">#770000ff</drawable>
<drawable name="green">#7700ff00</drawable>
<color name="solid_red">#f00</color>
<color name="solid_blue">#0000ff</color>
<color name="solid_green">#f0f0</color>
<color name="solid_yellow">#ffffff00</color>
</resources>
```

ANDROID ACTIVITIES

Method	Description	Next
onCreate()	Called when the activity is first created.	onStart()
onRestart()	Called after your activity has been stopped, just prior to it being started again. Always followed by onStart()	onStart()
onStart()	Called when the activity is becoming visible to the user. Followed by onResume() if the activity comes to the foreground, or onStop() if it becomes hidden.	onResume()or onStop()
onResume()	Called when the activity will start interacting with the user. Always followed by onPause().	onPause()
onPause()	Called when the system is about to start resuming a previous activity.	onResume() or onStop()
onStop()	Called when the activity is no longer visible to the user. Followed by either onRestart() if this activity is coming back to interact with the user, or onDestroy() if this activity is going away.	onRestart() or onDestroy()
onDestroy()	The final call you receive before your activity is destroyed.	nothing

KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM

DEPARTMENT OF INFORMATION TECHNOLOGY

20IT601PE –MOBILE APPLICATION DEVELOPMENT (KNCETR-2020) HANDOUT

UNIT II - USE INTERFACE TOOLS

Creating user interface – Views - Creating views – Layouts – Draw able resources – Resolution and density independence – Menus – Intents – Adapters – Using Internet resources – Dialogs.

CREATING USER INTERFACE

- The role of the UI is to display the application data on the screen and also to serve as the primary point of user interaction.
- User Interface (UI) design in Android is a graphical representation of views displayed on a smartphone or tablet. It allows users to interact with the features, functions, and contents of the application.
- To design UI, you need no prior programming knowledge, although it is nice to have web developing skills or programming skills.

Components used for Android application

- Main Action Bar (MAR)
- Split Action Bar (SAB)
- Content Area

Steps to follow in UI Design

- Download Android studio from your browser.
- Install Android studio.
- Open Android studio.
- Create a new project and select project type.
- Give it a name of your choice.
- Click on the resources folder.
- Click on the layout folder and begin to design your UI.

VIEWS COMPONENTS

- Views are used to customize User Interface (UI) design. A view is considered a building block for a proper User Interface created from the view class.
- A view can also be defined as a small rectangular box in Android development that responds to the user inputs, for example, buttons, checkboxes, etc.
- View group is an invisible container of other views, such as child view and other view groups.
- This view class is a super class of all the GUI components in Android.

- Views play a key role in the development of an application because you need settings present in the application, e.g.,color, style, themes, etc.

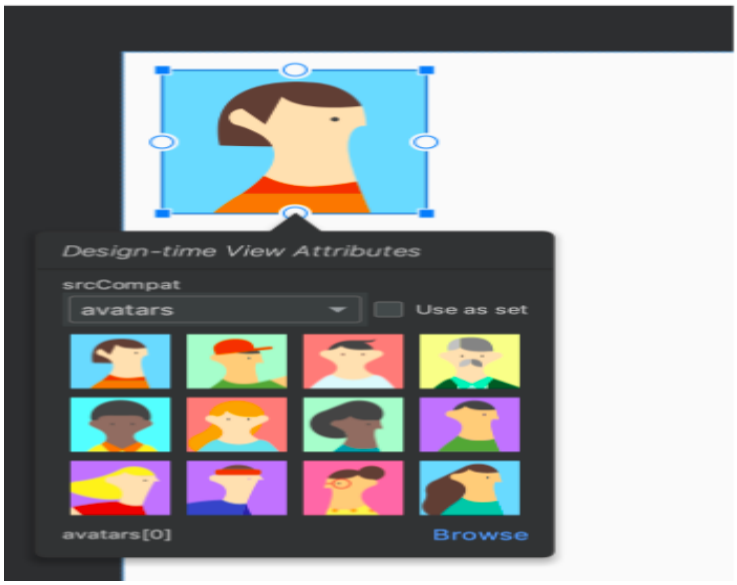
A simple UI Design

Layout as a subclass of View group

- The Layout defines the visual structures of the UI of user application.
- All elements in the layout are built using a hierarchy of views and view group object.
- The layout can be declared through a programming language, or through a simple XML layout file located in the resource layout folder of any project you work on.

Layout

- Layouts are kept in the folder called resources. Android studio creates a default XML layout file in the resources layout folder that is extremely useful if you know the basic at the time of compiling.



Advantages of XML Layout

- It is an immensely popular and widely used format.
- It helps provide the UI component from the logic, which in turn provides the flexibility to change one component without affecting the other.
- It is much easier to generate than writing direct code. It allows an easier drag and drop to generate interfaces for the Android application.

Types of layout:

- **Linear layout:** It is used to arrange views in a single column or row. It is further divided into a vertical or horizontal linear layout.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >
 <Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello, I am a Button" />
</LinearLayout>
```

- **Frame layout:** It is a placeholder on the screen used to display a single screen.

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
```

```
android:orientation="vertical" >
 <Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello, I am a Button" />
</FrameLayout>
```

- **Relative layout:** It enables you to specify how child views are positioned to each other.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
>
 <Button
android:id="@+id/button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello, I am a Button" />
</RelativeLayout>
```

- **Constraint layout:** It provides you with adaptable and flexible ways to create views for your applications.

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent“
 <TextView
android:id="@+id/text"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello, I am a TextView" />
 </androidx.constraintlayout.widget.ConstraintLayout>
```

INTENT

Intent is used to invoke components. It is mainly used to:

- Start the service
- Launch an activity

- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.
- **Android Intent is the message that is passed between components such as activities, content providers, broadcast receivers, services etc.**
- It is generally used with startActivity() method to invoke activity, broadcast receivers etc.
- The **dictionary meaning of intent is intention or purpose. So, it can be described as the intention to do action.**
- The LabeledIntent is the subclass of android.content.Intent class.

Intent Types

- There are two types of intents in android: implicit and explicit.

1) Implicit Intent

- **Implicit Intent doesn't specify the component. In such case, intent provides information of available components provided by the system that is to be invoked.**
- For example, you may write the following code to view the webpage.
- Intent intent=**new Intent(Intent.ACTION_VIEW);**
- intent.setData(Uri.parse("http://www.javatpoint.com"));
- startActivity(intent);

2) Explicit Intent

- Explicit Intent specifies the component. In such case, intent provides the external class to be invoked.

  Intent i = **new Intent(getApplicationContext(), ActivityTwo.class);**
  startActivity(i);

- **Android Implicit Intent Example**
- Let's see the simple example of implicit intent that displays a web page.
- ***activity_main.xml***
- *File: activity_main.xml*
- **<RelativeLayout xmlns:androclass="http://schemas.android.com/apk/res/android"**
- xmlns:tools="http://schemas.android.com/tools"
- android:layout_width="match_parent"
- android:layout_height="match_parent"
- tools:context=".MainActivity" >
- **<EditText**

- android:id="@+id/editText1"
- android:layout_width="wrap_content"
- android:layout_height="wrap_content"
- android:layout_alignParentTop="true"
- android:layout_centerHorizontal="true"
- android:layout_marginTop="44dp"
- **Android Implicit Intent Example**
- android:ems="10" />
- **<Button**
- android:id="@+id/button1"
- android:layout_width="wrap_content"
- android:layout_height="wrap_content"
- android:layout_below="@+id/editText1"
- android:layout_centerHorizontal="true"
- android:layout_marginTop="54dp"
- android:text="Visit" />
- </RelativeLayout>

Activity class

File: MainActivity.java

package org.sssit.implicitintent;

import android.net.Uri;

import android.widget.EditText;

public class MainActivity extends Activity {

@Override

protected void onCreate(Bundle savedInstanceState) {

super.onCreate(savedInstanceState);

setContentView(R.layout.activity_main);

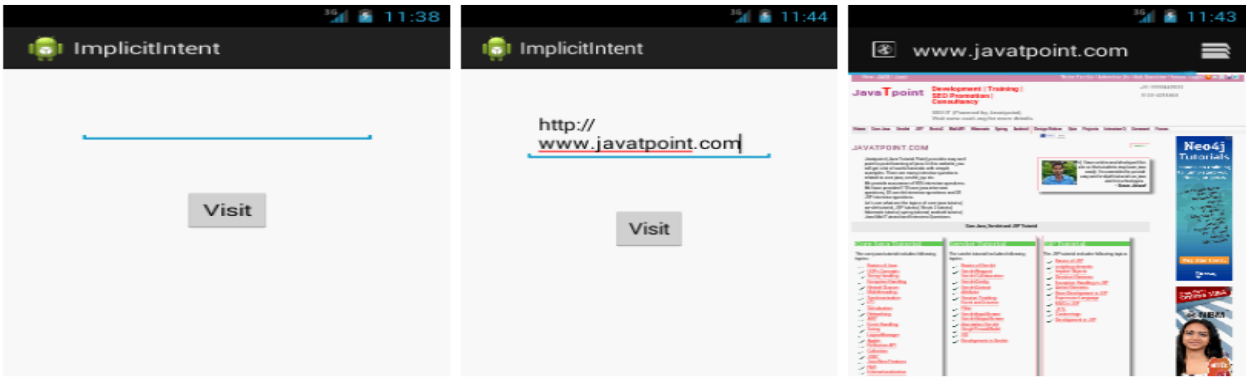
final EditTexteditText1=(EditText)findViewById(R.id.editText1);

Button button1=(Button)findViewById(R.id.button1);

- button1.setOnClickListener(new OnClickListener() {
- @Override
- public void onClick(View arg0) {
- String url=editText1.getText().toString();
- Intent intent=new Intent(Intent.ACTION_VIEW,Uri.parse(url));

- startActivity(intent);
- } }); } }

Output:



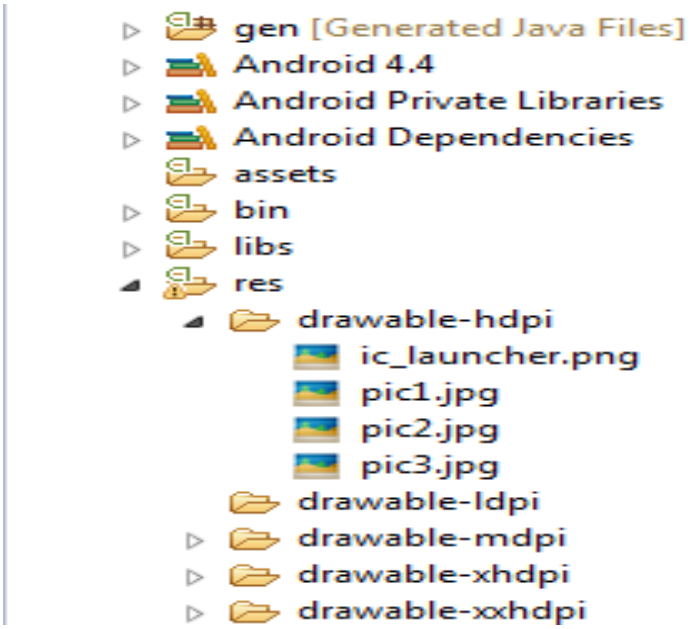
DRAWABLE RESOURCES

- **drawable/**
- For bitmap files (PNG, JPEG, or GIF), 9-Patch image files, and
- XML files that describe Drawable shapes or Drawable objects that
- contain multiple states (normal, pressed, or focused).
- res — This folder contains all the resources used in your application. It also contains a few other subfolders: drawable-<resolution>, layout, and values.

Android multiple screen: Screen Density

- We considered only the screen size but as we said earlier there are different densities too.
- Density affects primary the images so an image could be the right size one a screen device and looks too small in another device with more density.
- So we have to consider this problem.

Density	Android name
Small	around 120dpi (**ldpi)**
Normal	around 160dpi (**mdpi)**
High	around 240dpi (hdpi)
x-High	around 320dpi (**xhdpi)**



Option Menu with Icon

You need to have icon images inside the res/drawable directory. The android:icon element is used to display the icon on the option menu. You can write the string information in the strings.xml file. But we have written it inside the menu_main.xml file.

Option Menu with Icon

File: menu_main.xml

```
<menu xmlns:androclass="http://schemas.android.com/apk/res/android" >
<item android:id="@+id/item1"
android:icon="@drawable/add"
android:title="Item 1"/>
<item android:id="@+id/item2"
android:icon="@drawable/minus"
android:title="Item 2"/>
<item android:id="@+id/item3"
android:icon="@drawable/delete"
android:title="Item 3"/>
</menu>
```

- **Activity**

  An activity is a class that represents a single screen.

  It is like a Frame in AWT.

- **View**

A view is the UI element such as button, label, text field etc. Anything that you see is a view.

- **Intent**

Intent is used to invoke components. It is mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Service

- Service is a background process that can run for a long time.
- There are two types of services local and remote.
- Local service is accessed from within the application whereas remote service is accessed remotely from other applications running on the same device.

Content Provider

- Content Providers are used to share data between the applications.
- **Fragment**
- Fragments are like parts of activity.
- An activity can display one or more fragments on the screen at the same time.
- **MENU**
- In android, Options Menu is a primary collection of menu items for an activity and it is useful to implement actions that have a global impact on the app, such as Settings, Search, etc.
- In case, if we define items for the options menu in both activity or fragment, then those items will be combine and display in UI.

Adapter

- In Android, Adapter is a bridge between UI component and data source that helps us to fill data in UI component.
- It holds the data and send the data to an Adapter view then view can takes the data from the adapter view and shows the data on different views like as ListView, GridView,

Spinner etc.

- For more customization in Views we uses the base adapter or custom adapters.

Adapters In Android:

- There are the some commonly used Adapter in Android used to fill the data in the UI components.
- BaseAdapter – It is parent adapter for all other adapters
- ArrayAdapter – It is used whenever we have a list of single items which is backed by an array
- Custom ArrayAdapter – It is used whenever we need to display a custom list
- SimpleAdapter – It is an easy adapter to map static data to views defined in your XML file
- Custom SimpleAdapter – It is used whenever we need to display a customized list and needed to access the child items of the list or grid.

1. BaseAdapter In Android:

- BaseAdapter is a common base class of a general implementation of an Adapter that can be used in ListView, GridView, Spinner etc.
- Whenever we need a customized list in a ListView or customized grids in a GridView we create our own adapter and extend base adapter in that.
- Base Adapter can be extended to create a custom Adapter for displaying a custom list item. ArrayAdapter is also an implementation of BaseAdapter.

Custom Adapter code which extends the BaseAdapter in that:

```
public class CustomAdapter extends BaseAdapter {
@Override
public int getCount() {
return 0; }
@Override
public Object getItem(int i) {
return null; }
@Override
public long getItemId(int i) {
return 0; }
@Override
public View getView(int i, View view, ViewGroup viewGroup) { return null; }
```

2. ArrayAdapter In Android:

- Whenever we have a list of single items which is backed by an Array, we can use

ArrayAdapter. For instance, list of phone contacts, countries or names.

- ArrayAdapter(Context context, int resource, int textViewResourceId, T[] objects)

3. Custom ArrayAdapter In Android:

- ArrayAdapter is also an implementation of BaseAdapter, so if we want more customization then we can create a custom adapter and extend ArrayAdapter in that.

Below Custom adapter class MyAdapter extends ArrayAdapter in that:

```
public class MyAdapter extends ArrayAdapter
{
public MyAdapter(Context context, int resource, int textViewResourceId, List objects)
{ super(context, resource, textViewResourceId, objects);
}
@Override
public int getCount()
{
return super.getCount(); }
@Override
public View getView(int position, View convertView, ViewGroup parent)
{
return super.getView(position, convertView, parent);
} }
```

4. SimpleAdapter In Android:

- In Android SimpleAdapter is an easy Adapter to map static data to views defined in an XML file(layout).
- In Android we can specify the data backing to a list as an ArrayList of Maps(i.e. hashmap or other). Each entry in a ArrayList is corresponding to one row of a list.
- The Map contains the data for each row. Here we also specify an XML file(custom list items file) that defines the views which is used to display the row, and a mapping from keys in the Map to specific views.

DIALOG

- A dialog is a small window that prompts the user to make a decision or enter additional information.
- A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.

- The Dialog class is the base class for dialogs, but you should avoid instantiating Dialog directly. Instead, use one of the following subclasses:
- · AlertDialog : A dialog that can show a title, up to three buttons, a list of selectable items, or a custom layout.
- · DatePickerDialog or TimePickerDialog : A dialog with a pre-defined UI that allows the user to select a date or time.

KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM

DEPARTMENT OF INFORMATION TECHNOLOGY

20IT601PE –MOBILE APPLICATION DEVELOPMENT (KNCETR-2020)

HANDOUT

UNIT III - FILES, SAVING STATES AND DATABASES

Saving Simple Application Data – Creating and saving preferences – Preferences activity – Saving activity state - Loading files – File management tools - Introducing Android databases –SQLite – Cursors and content Values – Working with SQlite Databases – Creating and Using Content providers – Native Android content Providers.

SAVING SIMPLE APPLICATION DATA

Creating and saving preferences

Android provides many ways of storing data of an application. One of this way is called Shared Preferences. Shared Preferences allow you to save and retrieve data in the form of key,value pair.

In order to use shared preferences, you have to call a method getSharedPreferences() that returns a SharedPreference instance pointing to the file that contains the values of preferences.

SharedPreferences sharedpreferences = getSharedPreferences(MyPREFERENCES, Context.MODE_PRIVATE);

Shared Preferences are suitable for different situations. For example, when the user's settings need to be saved or to store data that can be used in different activities within the app. As you know, onPause() will always be called before your activity is placed in the background or destroyed, So for the data to be saved persistently, it's preferred to save it in onPause(), which could be restored in onCreate() of the activity. The data stored using shared preferences are kept private within the scope of the application. However, shared preferences are different from that activity's instance state.

Shared Preferences	Saved Instance State
Persist Data across user sessions, even if the app is killed and restarted, or the device is rebooted	Preserves state data across activity instances in the same user session.
Data that should be remembered across sessions, such as the user's preferred settings or game score.	Data that should not be remembered across sessions, such as the currently selected tab or current state of activity.

A common use is to store user preferences	A common use is to recreate the state after the device has been rotated

Nested classes of Shared Preferences

1. SharedPreferences.Editor: Interface used to write(edit) data in the SP file. Once editing has been done, one must commit() or apply() the changes made to the file.
2. SharedPreferences.OnSharedPreferenceChangeListener(): Called when a shared preference is changed, added, or removed. This may be called even if a preference is set to its existing value. This callback will be run on your main thread.

Following are the methods of Shared Preferences

1. contains(String key): This method is used to check whether the preferences contain a preference.
2. edit(): This method is used to create a new Editor for these preferences, through which you can make modifications to the data in the preferences and atomically commit those changes back to the SharedPreferences object.
3. getAll(): This method is used to retrieve all values from the preferences.
4. getBoolean(String key, boolean defValue): This method is used to retrieve a boolean value from the preferences.
5. getFloat(String key, float defValue): This method is used to retrieve a float value from the preferences.
6. getInt(String key, int defValue): This method is used to retrieve an int value from the preferences.
7. getLong(String key, long defValue): This method is used to retrieve a long value from the preferences.
8. getString(String key, String defValue): This method is used to retrieve a String value from the preferences.
9. getStringSet(String key, Set defValues): This method is used to retrieve a set of String values from the preferences.
10. registerOnSharedPreferencechangeListener(SharedPreferences.OnSharedPreferencechangeListener listener): This method is used to register a callback to be invoked when a

change happens to a preference.

11. unregisterOnSharedPreferencechangeListener(SharedPreferences.OnSharedPreferencechangeListener listener): This method is used to unregister a previous callback.

Following is a sample byte code on how to write Data in Shared Preferences:

```
// Storing data into SharedPreferences

SharedPreferences sharedPreferences =
getSharedPreferences("MySharedPref",MODE_PRIVATE);

// Creating an Editor object to edit(write to the file)

SharedPreferences.Editor myEdit = sharedPreferences.edit();

// Storing the key and its value as the data fetched from edittext

myEdit.putString("name", name.getText().toString());

myEdit.putInt("age", Integer.parseInt(age.getText().toString()));

// Once the changes have been made, we need to commit to apply those changes made,

// otherwise, it will throw an error

myEdit.commit();
```

Preferences activity – Saving activity state

Example to Demonstrate the use of Shared Preferences in Android

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity"

    tools:ignore="HardcodedText">

      <TextView

        android:id="@+id/textview"
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="32dp"
    android:text="Shared Preferences Demo"
    android:textColor="@android:color/black"
    android:textSize="24sp" />
<!--EditText to take the data from the user and save the data in SharedPreferences-->
<EditText
    android:id="@+id/edit1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@+id/textview"
    android:layout_marginStart="16dp"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="16dp"
    android:hint="Enter your Name"
    android:padding="10dp" />
<!--EditText to take the data from the user and save the data in SharedPreferences-->
<EditText
    android:id="@+id/edit2"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@+id/edit1"
    android:layout_marginStart="16dp"
    android:layout_marginTop="8dp"
```

```
    android:layout_marginEnd="16dp"

    android:hint="Enter your Age"

    android:inputType="number"

    android:padding="10dp" />

</RelativeLayout>
```

Working with the MainActivity file to handle the two of the EditText to save the data entered by the user inside the SharedPreferences.

Below is the code for the MainActivity file. Comments are added inside the code to understand the code in more detail.

```
import androidx.appcompat.app.AppCompatActivity;

import android.content.SharedPreferences;

import android.os.Bundle;

import android.widget.EditText;

public class MainActivity extends AppCompatActivity {

   private EditText name, age;

   @Override

   protected void onCreate(Bundle savedInstanceState) {

      super.onCreate(savedInstanceState);

      setContentView(R.layout.activity_main);

      name = findViewById(R.id.edit1);

      age = findViewById(R.id.edit2);

   }

   // Fetch the stored data in onResume() Because this is what will be called when the app opens again

   @Override

   protected void onResume() {

      super.onResume();
```

```
        // Fetching the stored data from the SharedPreference

        SharedPreferences sh = getSharedPreferences("MySharedPref", MODE_PRIVATE);

        String s1 = sh.getString("name", "");

        int a = sh.getInt("age", 0);

        // Setting the fetched data in the EditTexts

        name.setText(s1);

        age.setText(String.valueOf(a));

    }

    // Store the data in the SharedPreference in the onPause() method

    // When the user closes the application onPause() will be called and data will be stored

    @Override

    protected void onPause() {

        super.onPause();

        // Creating a shared pref object with a file name "MySharedPref" in private mode

        SharedPreferences sharedPreferences = getSharedPreferences("MySharedPref",
MODE_PRIVATE);

        SharedPreferences.Editor myEdit = sharedPreferences.edit();

       // write all the data entered by the user in SharedPreference and apply

        myEdit.putString("name", name.getText().toString());

        myEdit.putInt("age", Integer.parseInt(age.getText().toString()));

        myEdit.apply();

    }}
```

FILE MANAGEMENT TOOLS

File management tools are utility software that manages files of the computer system. Since files are an important part of the system as all the data is stored in the files. Therefore, this utility software help to browse, search, arrange, find information, and quickly preview the files of the system.

A utility software or system utilities is a type of system software that helps in the proper and smooth functioning of a computer system. Moreover, they assist the operating system to manage, organize, maintain and optimize the functioning of a computer system. A file management system has a limited amount of capabilities. Furthermore, they work on files or a group of files. We can also call them file manager.

Windows Explorer is a default file management tool present in the system. Some other examples of file management tools are Google Desktop, Double Commander, Directory Opus, etc.

Functions of the File Management System

Functions of a file management system are as follows:

- Store, arrange, or accessing files on a disk or other storage locations.
- Creating new files.
- Displaying the old files.
- Adding and editing the data in files.
- Moving files from one location to another.
- Sorting files according to the given criteria. For example, file size, file location, modified date, creation date, etc.

Features of the File Management System

A file management system has the following features:

- Arranging the files and folders hierarchically.
- Report generation
- Notes
- Status
- Assigning documents for processing in a queue.
- Add or edit metadata of files.
- Create, modify, delete, or manage other file operations.

- Simple interact to access and manage files.
- Managing different types of files with extensions .xls, .pdf, .doc etc

Windows Explorer: Default File Management Tool

Windows Explorer is a file utility program included in windows explorer with the name file explorer. a file management system which is the default management tool. It allows the user to view, access, manage, or delete files on your system using a simple window. Functions and features of windows explore are as follows:

- It can access and manage files of different software namely, excel, word, etc
- Helps to browse files on your hard drive and also, external storage like USB.
- It helps to manage files in a hierarchy in folders.
- Moreover, it also helps to drag and move files among folders.

The default file management tool in mac devices is Finder. Although Windows Explorer and Finder have many features, they are not enough for business purposes.

Examples of File Management Tool

Certain examples are as follows:

- eFileCabinet
- Microsoft OneDrive
- Dropbox
- Apple iCloud
- Google Drive

DATABASES

- SQLite Database in Android.
- Database - Package
- The main package is android.database.sqlite that contains the classes to manage your own databases.

UNDERSTANDING OF SQLite DATABASE

- SQLite is a opensource SQL database that stores data to a text file on a device.
- Android comes in with built in SQLite database implementation.
- SQLite supports all the relational database features.

Why using SQLite Database:

- ❖ SQLite is lightweight process.
- ❖ No connectivity is needed.
- ❖ For example: In order to access this database, you don't need to establish any kind of connections for it like JDBC,ODBC e.t.c.

DATABASE – CREATION

- In order to create a database you just need to call this method openOrCreateDatabase with your database name and mode as a parameter.
- It returns an instance of SQLite database which you have to receive in your own object.

- Its syntax is given below

  SQLiteDatabase mydatabase = OpenOrCreateDatabase("your database name",MODE_PRIVATE,null);

Database – Insertion

- we can create table or insert data into table using execSQL method defined in SQLiteDatabase class.
- Its syntax is given below
- mydatabase.execSQL("CREATE TABLE IF NOT EXISTS TutorialsPoint(Username VARCHAR, Password VARCHAR);");
- mydatabase.execSQL("INSERT INTO TutorialsPoint VALUES('admin','admin');");

Database – Fetching

- We can retrieve anything from database using an object of the Cursor class. We will call a method of this class called rawQuery and it will return a resultset with the cursor pointing to the table. We can move the cursor forward and retrieve the data.

  Cursor resultSet = mydatbase.rawQuery("Select * from MRCET",null);

  resultSet.moveToFirst();

  String username = resultSet.getString(0);

  String password = resultSet.getString(1);

Database - Helper class

- For managing all the operations related to the database, an helper class has been given and is called SQLiteOpenHelper.
- It automatically manages the creation and update of the database.
- To experiment with this example, you need to run this on an actual device on which camera is supported.

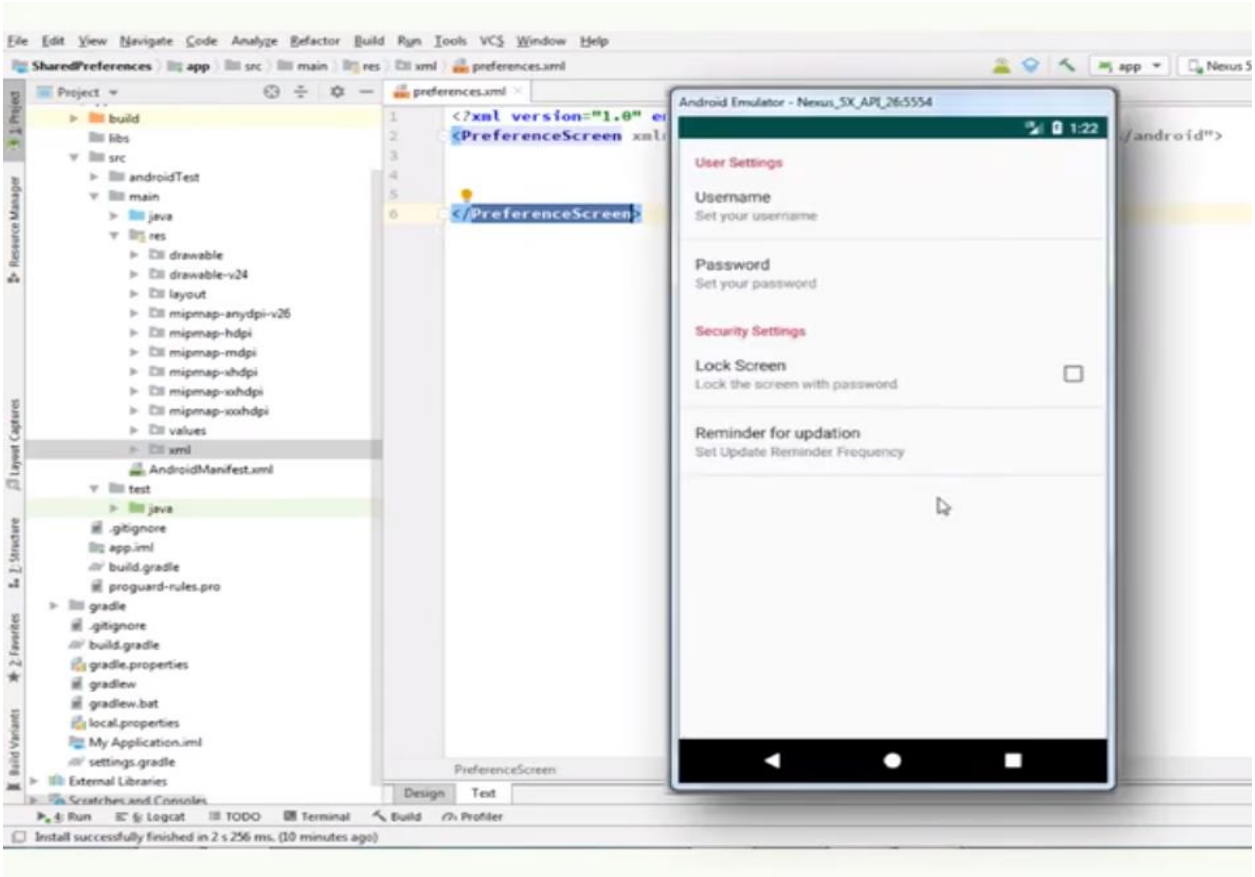
```
public class DBHelper extends SQLiteOpenHelper {

public DBHelper(){

super(context,DATABASE_NAME,null,1);

}
```

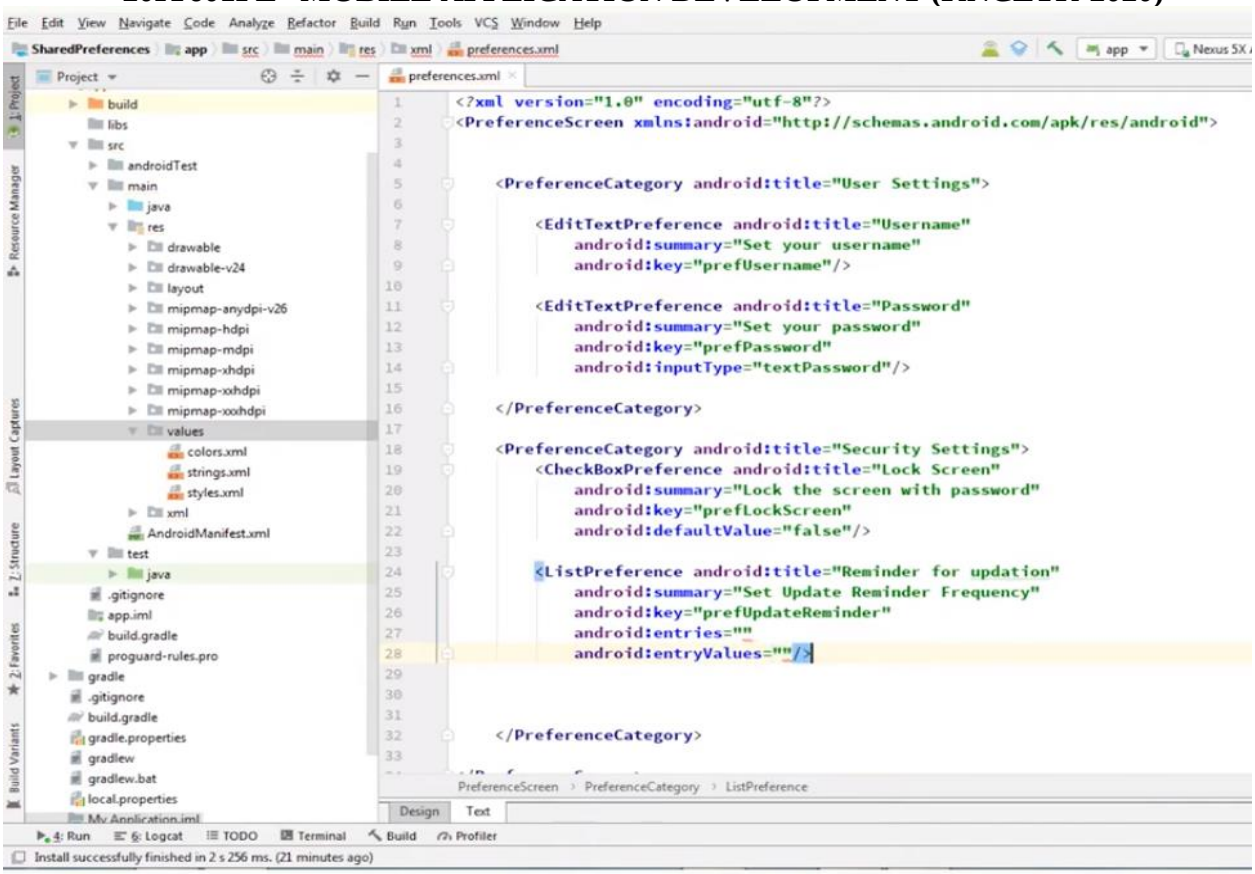
```
public void onCreate(SQLiteDatabase db) {}

public void onUpgrade(SQLiteDatabase database, int oldVersion, int newVersion) {}

}
```

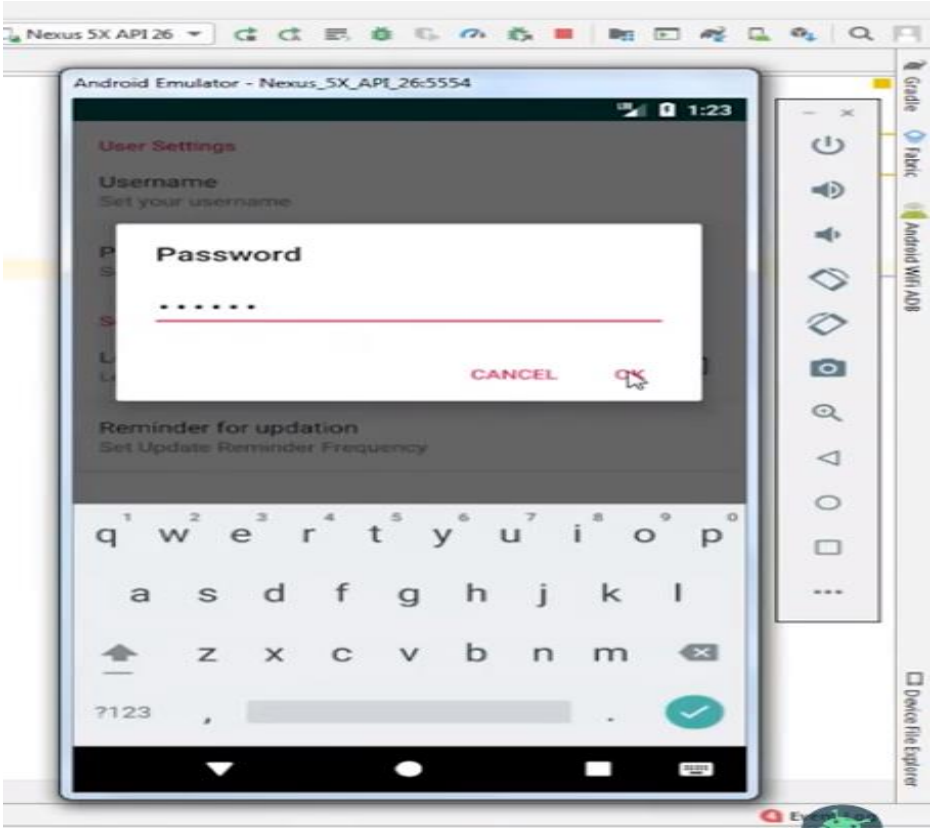
Example

- Demonstrating the use of SQLite Database.
- It creates a basic contacts applications that allows insertion, deletion and modification of contacts.

Implementing preference activity




File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
SharedPreferences app src main res xml preferences.xml
Project
preferences.xml
<?xml version="1.0" encoding="utf-8"?>
<PreferenceScreen xmlns:android="http://schemas.android.com/apk/res/android">
<PreferenceCategory android:title="User Settings">
<EditTextPreference android:title="Username"
android:summary="Set your username"
android:key="prefUsername"/>
<EditTextPreference android:title="Password"
android:summary="Set your password"
android:key="prefPassword"
android:inputType="textPassword"/>
</PreferenceCategory>
<PreferenceCategory android:title="Security Settings">
<CheckBoxPreference android:title="Lock Screen"
android:summary="Lock the screen with password"
android:key="prefLockScreen"
android:defaultValue="false"/>
<ListPreference android:title="Reminder for updation"
android:summary="Set Update Reminder Frequency"
android:key="prefUpdateReminder"
android:entries=""
android:entryValues=""/>
</PreferenceCategory>
PreferenceScreen › PreferenceCategory › ListPreference
Design Text
Install successfully finished in 2 s 256 ms. (21 minutes ago)


Nexus 5X API 26
Android Emulator - Nexus_5X_API_26:5554
1:23
User Settings
Username
Set your username
Password
CANCEL
OK
Reminder for updation
Set Update Reminder Frequency

CREATING AND USING CONTENT PROVIDERS

CONTENT PROVIDERS IN ANDROID

In Android, Content Providers are a very important component that serves the purpose of a relational database to store the data of applications. The role of the content provider in the android system is like a central repository in which data of the applications are stored, and it facilitates other applications to securely access and modifies that data based on the user requirements. Android system allows the content provider to store the application data in several ways. Users can manage to store the application data like images, audio, videos, and personal contact information by storing them in SQLite Database**, in files**, **or even on a network**. In order to share the data, content providers have certain permissions that are used to grant or restrict the rights to other applications to interfere with the data.

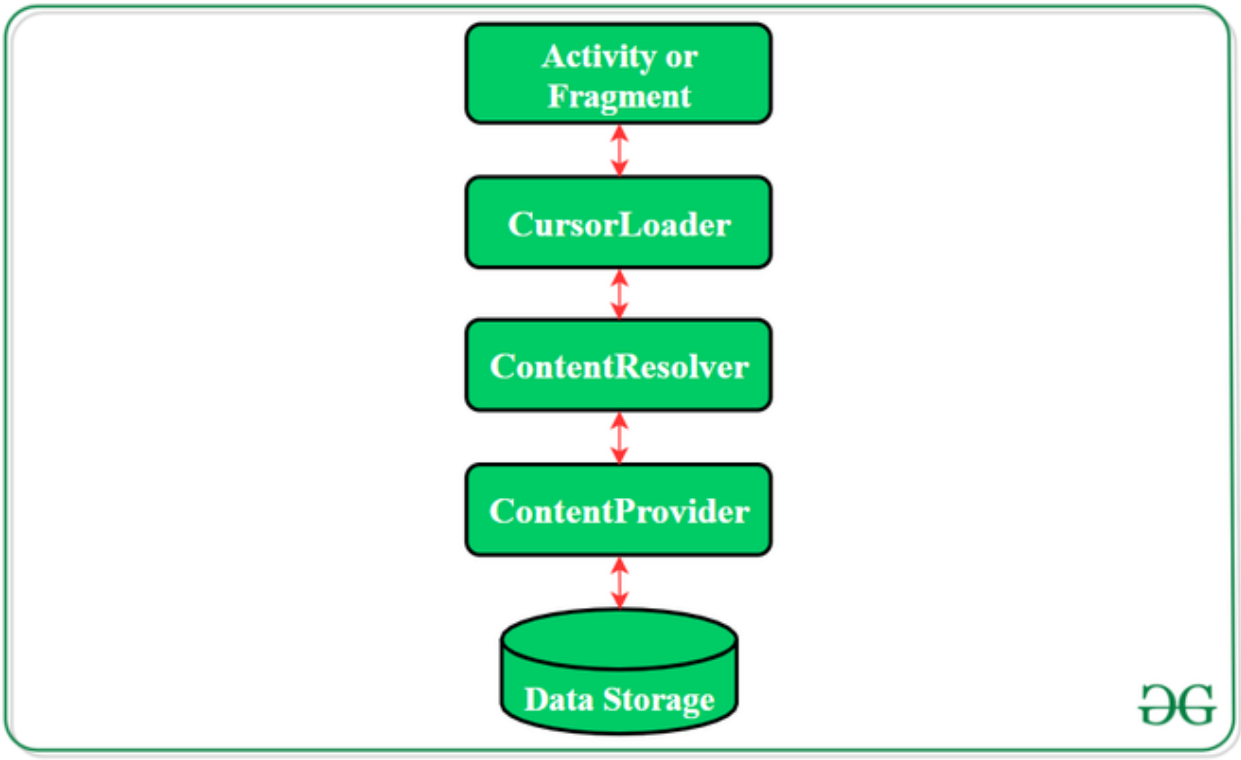
Operations in Content Provider

Four fundamental operations are possible in Content Provider namely **Create**, **Read**, **Update**, and **Delete**. These operations are often termed as **CRUD operations**.

- **Create:** Operation to create data in a content provider.
- **Read:** Used to fetch data from a content provider.
- **Update:** To modify existing data.
- **Delete:** To remove existing data from the storage.

Working of the Content Provider

UI components of android applications like Activity and Fragments use an object **CursorLoader** to send query requests to **ContentResolver.** The ContentResolver object sends requests (like create, read, update, and delete) to the **ContentProvider** as a client. After receiving a request, ContentProvider process it and returns the desired result.



Creating a Content Provider

Following are the steps which are essential to follow in order to create a Content Provider:

- Create a class in the same directory where the **MainActivity** file resides and this class must extend the ContentProvider base class.
- To access the content, define a content provider URI address.
- Create a database to store the application data.
- Implement the **six abstract methods** of ContentProvider class.
- Register the content provider in **AndroidManifest.xml** file using **<provider> tag**.

Six abstract methods of ContenProvider class:

Abstract Method	**Description**
query()	A method that accepts arguments and fetches the data from the desired table. Data is retired as a cursor object.
insert()	To insert a new row in the database of the content provider. It returns the content URI of the inserted row.

Abstract Method	Description
update()	This method is used to update the fields of an existing row. It returns the number of rows updated.
delete()	This method is used to delete the existing rows. It returns the number of rows deleted.
getType()	This method returns the Multipurpose Internet Mail Extension(MIME) type of data to the given Content URI.
onCreate()	As the content provider is created, the android system calls this method immediately to initialise the provider.

Android Internal Storage

In android, we have different storage options such as shared preferences, internal storage, external storage, SQLite storage, etc. to store and retrieve the application data based on our requirements.

To use the **Internal Storage** option to store and retrieve the data from the device's internal memory.

In android, **Internal Storage** is useful to store the data files locally on the device's internal memory using a **FileOutputStream** object. After storing the data files in device internal storage, we can read the data file from the device using a **FileInputStream** object.

The data files saved in the internal are managed by an android framework and it can be accessed anywhere within the app to read or write data into the file, but it's not possible to access the file from any other app so it's secured. When the user uninstalls the app, automatically these data files will be removed from the device internal storage.

Write a File to Internal Storage

By using the android **FileOutputStream** object **openFileOutput** method, we can easily create and write data to a file in a device's internal storage.

Following is the code snippet to create and write a private file to the device's internal memory.

```
String FILENAME = "user_details";
String name = "suresh";
FileOutputStream fstream = openFileOutput(FILENAME, Context.MODE_PRIVATE);
fstream.write(name.getBytes());
fstream.close();
```

If you observe above code, we are creating and writing a file in device internal storage by using FileOutputStream object openFileOutput method
with filename and MODE_PRIVATE mode to make the file private to our application. We used **write()** method to write the data in file and used **close()** method to close the stream.

In android, we have different modes such as **MODE_APPEND**, **MODE_WORLD_READBLE**, **MODE_WORLD_WRITEABLE**, etc. to use it in our application based on your requirements.

Read a File from Internal Storage

By using the android **FileInputStream** object **openFileInput** method, we can easily read the file from the device's internal storage.

Following is the code snippet to read data from a file that is in the device's internal memory.

```
String FILENAME = "user_details";
FileInputStream fstream = openFileInput(FILENAME);
StringBuffer sbuffer = new StringBuffer();
int i;
while ((i = fstream.read())!= -1){
   sbuffer.append((char)i);
}
fstream.close();
```

If you observe above code, we are reading a file from device internal storage by using FileInputStream object openFileInput method with file name. We used read() method to read one character at a time from the file and used close() method to close the stream.

Android Internal Storage Example

Following is the example of storing and retrieving the data files from the device's internal memory by using **FileOutputStream** and **FileInputStream** objects.

Create a new android application using android studio and give names as **InternalStorageExample**. In case if you are not aware of creating an app in android studio check this article Android Hello World App.

Once we create an application, open **activity_main.xml** file from **\res\layout** folder path and write the code like as shown below.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="vertical" android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
      android:id="@+id/fstTxt"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginLeft="100dp"
      android:layout_marginTop="150dp"
      android:text="UserName" />
   <EditText
      android:id="@+id/txtName"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginLeft="100dp"
      android:ems="10"/>
   <TextView
      android:id="@+id/secTxt"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Password"
      android:layout_marginLeft="100dp" />
   <EditText
      android:id="@+id/txtPwd"
      android:inputType="textPassword"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_marginLeft="100dp"
      android:ems="10" />
   <Button
```

```
    android:id="@+id/btnSave"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="100dp"
    android:text="Save" />
</LinearLayout>
```

Now we will create another layout resource file **details.xml** in **\res\layout** path to get the first activity (**activity_main.xml**) details in second activity file for that right click on your **layout** folder Go to **New** select **Layout Resource File** and give name as **details.xml**.

Once we create a new layout resource file **details.xml**, open it and write the code like as shown below:

details.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="vertical" android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:id="@+id/resultView"
      android:layout_gravity="center"
      android:layout_marginTop="170dp"
      android:textSize="20dp"/>
   <Button
      android:id="@+id/btnBack"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_gravity="center"
      android:layout_marginTop="20dp"
      android:text="Back" />
</LinearLayout>
```

File **MainActivity.java** from **\java\com.tutlane.internalstorageexample** path and write the code like as shown below:

MainActivity.java

```
package com.tutlane.internalstorageexample;
import android.content.Context;
import android.content.Intent;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
public class MainActivity extends AppCompatActivity {
    EditText uname, pwd;
    Button saveBtn;
    FileOutputStream fstream;
    Intent intent;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        uname = (EditText)findViewById(R.id.txtName);
        pwd = (EditText)findViewById(R.id.txtPwd);
        saveBtn = (Button)findViewById(R.id.btnSave);
        saveBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String username = uname.getText().toString()+"\n";
                String password = pwd.getText().toString();
                try {
                    fstream = openFileOutput("user_details", Context.MODE_PRIVATE);
                    fstream.write(username.getBytes());
                    fstream.write(password.getBytes());
                    fstream.close();
```

```
                Toast.makeText(getApplicationContext(), "Details Saved
Successfully",Toast.LENGTH_SHORT).show();
                intent = new Intent(MainActivity.this,DetailsActivity.class);
                startActivity(intent);
            } catch (FileNotFoundException e) {
                e.printStackTrace();
            } catch (IOException e) {
                e.printStackTrace();
            }  }
        });    }}
```

If you observe above code, we are taking entered username and password details and saving it in device local file and redirecting the user to another activity.

DetailsActivity.java

```
package com.tutlane.internalstorageexample;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
/**
 * Created by tutlane on 04-01-2018.
 */
public class DetailsActivity extends AppCompatActivity {
    FileInputStream fstream;
    Intent intent;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.details);
```

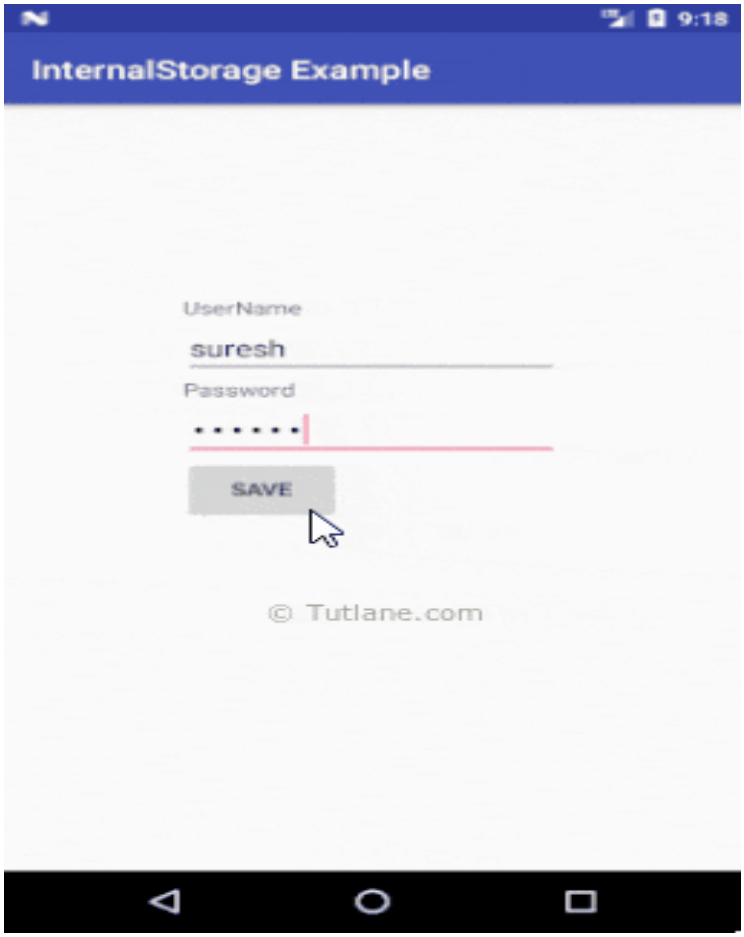
```
    TextView result = (TextView)findViewById(R.id.resultView);
    Button back = (Button)findViewById(R.id.btnBack);
    try {
        fstream = openFileInput("user_details");
        StringBuffer sbuffer = new StringBuffer();
        int i;
        while ((i = fstream.read())!= -1){
            sbuffer.append((char)i);
        }
        fstream.close();
        String details[] = sbuffer.toString().split("\n");
        result.setText("Name: "+ details[0]+"\nPassword: "+details[1]);
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
    back.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            intent = new Intent(DetailsActivity.this,MainActivity.class);
            startActivity(intent);
        }     });
}}
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.tutlane.internalstorageexample">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
```

```
    android:theme="@style/AppTheme">
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
<activity android:name=".DetailsActivity" android:label="Internal Storage - Details"></activity>
  </application>
</manifest>
```

Output of Android Internal Storage Example



Android External Storage

In android, we have different storage options such as shared preferences, internal storage, external storage, SQLite storage, etc. to store and retrieve the application data based on our requirements.

To use the External Storage option to store and retrieve the data from external storage media such as SD card.

In android, External Storage is useful to store the data files publically on the shared external storage using the FileOutputStream object. After storing the data files on external storage, we can read the data file from external storage media using a FileInputStream object.

The data files saved in external storage are word-readable and can be modified by the user when they enable USB mass storage to transfer files on a computer.

Grant Access to External Storage

To read or write files on the external storage, our app must acquire the WRITE_EXTERNAL_STORAGE and READ_EXTERNAL_STORAGE system permissions. For that, we need to add the following permissions in the android manifest file like as shown below.

```
<manifest>

 ....

   <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

   <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

   ....

</manifest>
```

Checking External Storage Availability

Before we do any work with external storage, first we need to check whether the media is available or not by calling getExternalStorageState(). The media might be mounted to a computer, missing, read-only or in some other state. To get the media status, we need to write the code like as shown below.

```
boolean Available= false;

boolean Readable= false;
```

```
String state = Environment.getExternalStorageState();

if(Environment.MEDIA_MOUNTED.equals(state)){

    // Both Read and write operations available

    Available= true;

} else if (Environment.MEDIA_MOUNTED_READ_ONLY.equals(state)){

    // Only Read operation available

    Available= true;

    Readable= true;

} else {

    // SD card not mounted

    Available = false;

}
```

KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM

DEPARTMENT OF INFORMATION TECHNOLOGY

20IT601PE –MOBILE APPLICATION DEVELOPMENT (KNCETR-2020)

HANDOUT

UNIT IV – HARDWARE SENSORS AND LOCATION-BASED SERVICES

Using Sensors and the Sensor Manager– Monitoring a Device's Movement and Orientation –Introducing the Environmental Sensors– Using Location-Based Services– Selecting a Location Provider –Finding Your Current Location.

USING SENSORS AND THE SENSOR MANAGER

The Sensor Manager is used to manage the sensor hardware available on Android devices. Use getSystemService to return a reference to the Sensor Manager Service, as shown in the following snippet:

```
String service_name = Context.SENSOR_SERVICE;
SensorManagersensorManager = (SensorManager)getSystemService(service_name);
```

The Sensor class includes a set of constants that describe which type of hardware sensor is being represented by a particular Sensor object. These constants take the form of Sensor.TYPE_<TYPE>.

The following section describes each supported sensor type, after which you'll learn how to find and use these sensors.

Supported Android Sensors

The following sections describe each sensor type currently available. Note that the hardware available on the host device determines which of these sensors will be available to your application.

- Sensor.TYPE_AMBIENT_TEMPERATURE — Introduced in Android 4.0 (API level 14) to replace the ambiguous — and deprecated — Sensor.TYPE_TEMPERATURE. This is a thermometer that returns the temperature in degrees Celsius; the temperature returned will be the ambient room temperature.
- Sensor.TYPE_ACCELEROMETER — A three-axis accelerometer that returns the current acceleration along three axes in m/s2 (meters per second, per second.) The accelerometer is explored in greater detail later in this chapter.
- Sensor.TYPE_GRAVITY — A three-axis gravity sensor that returns the current direction and magnitude of gravity along three axes in m/s2. The gravity sensor typically is implemented as a virtual sensor by applying a low-pass filter to the accelerometer sensor results.
- Sensor.TYPE_LINEAR_ACCELERATION — A three-axis linear acceleration Sensor that returns the acceleration, not including gravity, along three axes in m/s2. Like the gravity sensor, the linear acceleration typically is implemented as a virtual sensor using the

accelerometer output. In this case, to obtain the linear acceleration, a high-pass filter is applied to the accelerometer output.

- Sensor.TYPE_GYROSCOPE — A three-axis gyroscope that returns the rate of device rotation along three axes in radians/second. You can integrate the rate of rotation over time to determine the current orientation of the device; however, it generally is better practice to use this in combination with other sensors (typically the accelerometers) to provide asmoothed and corrected orientation.
- Sensor.TYPE_ROTATION_VECTOR — Returns the orientation of the device as a combination of an angle around an axis. It typically is used as an input to the getRotationMatrixFromVector method from the Sensor Manager to convert the returned rotation vector into a rotation matrix. The rotation vector Sensor typically is implemented as a virtual sensor that can combine and correct the results obtained from multiple sensors, such as the accelerometers and gyroscopes, to provide a smoother rotation matrix.
- Sensor.TYPE_MAGNETIC_FIELD — A magnetometer that finds the current magnetic field in microteslas (μT) along three axes.
- Sensor.TYPE_PRESSURE — An atmospheric pressure sensor, or barometer, that returns the current atmospheric pressure in millibars (mbars) as a single value. The pressure Sensor can be used to determine altitude using the getAltitude method on the Sensor Manager to compare the atmospheric pressure in two locations. Barometers can also be used in weather forecasting by measuring changes in atmospheric pressure in the same location.
- Sensor.TYPE_RELATIVE_HUMIDITY — A relative humidity sensor that returns the current relative humidity as a percentage. This Sensor was introduced in Android 4.0 (API level 14).
- Sensor.TYPE_PROXIMITY — A proximity sensor that indicates the distance between the device and the target object in centimeters. How a target object is selected, and the distances supported, will depend on the hardware implementation of the proximity detector. Some proximity sensors can return only “near” or “far” results, in which case the latter will be represented as the Sensor’s maximum range, and the former using any lower value. Typical uses for the proximity sensor are to detect when the device is being held up against the user’s ear, to automatically adjust screen brightness, or to initiate a voice command.
- Sensor.TYPE_LIGHT — An ambient light sensor that returns a single value describing the ambient illumination in lux. A light sensor commonly is used to control the screen brightness dynamically.

MONITORING A DEVICE'S MOVEMENT AND ORIENTATION

Accelerometers, compasses, and (more recently) gyroscopes offer the ability to provide functionality based on device direction, orientation, and movement. You can use these Sensors to offer new and innovative input mechanisms, in addition to (or as an alternative to) traditional touch screen, trackball, and keyboard input.

The availability of specific Sensors depends on the hardware platform on which your application runs. A 70" flat screen weighs more than 150 pounds, making it difficult to lift and awkward to maneuver. As a result Android-powered TVs are unlikely to include orientation or movement sensors — so it's good practice to offer users alternatives in case their devices don't support such Sensors.

Where they are available, movement and orientation sensors can be used by your application to:

- Determine the device orientation
- React to changes in orientation
- React to movement or acceleration
- Understand which direction the user is facing
- Monitor gestures based on movement, rotation, or acceleration

This opens some intriguing possibilities for your applications. By monitoring orientation, direction, and movement, you can:

1. Use the compass and accelerometers to determine your heading and orientation. Use these with a map, camera, and location-based service to create augmented-reality UIs that overlay location-based data over a real-time camera feed.
2. Create UIs that adjust dynamically as the orientation of the device changes. In the most simple case, Android alters the screen orientation when the device is rotated from portrait to landscape or vice versa, but applications such as the native Gallery use orientation changes to provide a 3D effect on stacks of photos.
3. Monitor for rapid acceleration to detect if a device has been dropped or thrown.
4. Measure movement or vibration. For example, you could create an application that lets a user lock his or her device; if any movement is detected while it's locked, it could send an alert SMS that includes the current location.
5. Create UI controls that use physical gestures and movement as input.

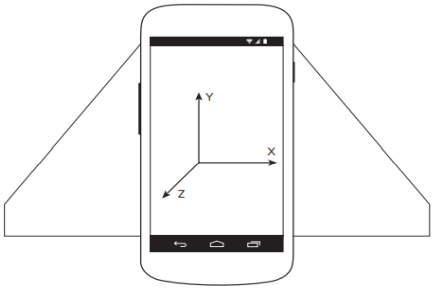
Determining the Natural Orientation of a Device

Before calculating the device's orientation, you must first understand its "at rest" (natural) orientation. The natural orientation of a device is the orientation in which the

orientation is 0 on all three axes. The natural orientation can be either portrait or landscape, but it typically is identifiable by the placement of branding and hardware buttons.

For a typical smartphone, the natural orientation is with the device lying on its back on a desk, with the top of the device pointing due north.

More creatively, you can imagine yourself perched on top of a jet fuselage during level flight. An Android device has been strapped to the fuselage in front of you. In its natural orientation the screen is pointing up into space, the top of the device pointing towards the nose of the plane, and the plane is heading due north, as shown in below Figure.



Example: Finding the screen orientation relative to the natural orientation

```
String windowSrvc = Context.WINDOW_SERVICE;
WindowManagerwm = ((WindowManager) getSystemService(windowSrvc));
Display display = wm.getDefaultDisplay();
int rotation = display.getRotation();
switch (rotation) {
case (Surface.ROTATION_0) : break; // Natural
case (Surface.ROTATION_90) : break; // On its left side
case (Surface.ROTATION_180) : break; // Upside down
case (Surface.ROTATION_270) : break; // On its right side
default: break;
}
```

Example: Monitoring the accelerometer and magnetometer

```
private float[] accelerometerValues;
private float[] magneticFieldValues;
finalSensorEventListenermyAccelerometerListener = new SensorEventListener() {
public void onSensorChanged(SensorEventsensorEvent) {
if (sensorEvent.sensor.getType() == Sensor.TYPE_ACCELEROMETER)
accelerometerValues = sensorEvent.values;
 }
```

```
public void onAccuracyChanged(Sensor sensor, int accuracy) {}
};
finalSensorEventListenermyMagneticFieldListener = new SensorEventListener() {
public void onSensorChanged(SensorEventsensorEvent) {
if (sensorEvent.sensor.getType() == Sensor.TYPE_MAGNETIC_FIELD)
magneticFieldValues = sensorEvent.values;
 }
public void onAccuracyChanged(Sensor sensor, int accuracy) {}
};
```

Register both Sensors with the Sensor Manager, as shown in the following code extension; this snippet uses the default hardware and UI update rate for both Sensors:

```
SensorManagersm = (SensorManager)getSystemService(Context.SENSOR_SERVICE);
Sensor aSensor = sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
Sensor mfSensor = sm.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD);
sm.registerListener(myAccelerometerListener,aSensor, SensorManager.SENSOR_DELAY_UI);
sm.registerListener(myMagneticFieldListener, mfSensor,
SensorManager.SENSOR_DELAY_UI);
```

To calculate the current orientation from these Sensor values, use the getRotationMatrix and getOrientation methods from the Sensor Manager, as shown below.

```
float[] values = new float[3];
float[] R = new float[9];
SensorManager.getRotationMatrix(R, null, accelerometerValues,
SensorManager.getOrientation(R, values);
// Convert from radians to degrees if preferred.
values[0] = (float) Math.toDegrees(values[0]); // Azimuth
values[1] = (float) Math.toDegrees(values[1]); // Pitch
values[2] = (float) Math.toDegrees(values[2]); // Roll
```

The order of the returned values is also different from the axes used by the accelerometer and magnetometer Sensors. Each resultis in radians, with positive values representing anticlockwise rotation around the axis:

- values[0] — The azimuth, or rotation around the z-axis, is zero when the device is heading magnetic north.
- values[1] — The pitch, or rotation around the x-axis.
- values[2] — The roll, or rotation around the y-axis.

INTRODUCING THE ENVIRONMENTAL SENSORS

Like orientation Sensors, the availability of specific environmental Sensors depends on the host hardware. Where they are available, environmental Sensors can be used by your application to:

- Improve location detection by determining the current altitude
- Track movements based on changes in altitude
- Alter the screen brightness or functionality based on ambient light
- Make weather observations and forecasts
- Determine on which planetary body the device is currently located

Using the Barometer Sensor

A barometer is used to measure atmospheric pressure. The inclusion of this sensor in some Android devices makes it possible for a user to determine his or her current altitude and, potentially, toforecast weather changes.

To monitor changes in atmospheric pressure, register an implementation of SensorEventListenerwith the Sensor Manager, using a Sensor object of type Sensor.TYPE_PRESSURE. The currentatmospheric pressure is returned as the fi rst (and only) value in the returned values array in hector-pascals (hPa), which is an equivalent measurement to millibars (mbar).

To ensure accurate results, you should use a local value for sea-level atmospheric pressure, although the Sensor Manager provides a value for one standard atmosphere via the PRESSURE_STANDARD_ATMOSPHERE constant as a useful approximation

To calculate the current altitude, you can use the static getAltitude method from the Sensor Manager, as shown below, supplying it with the current pressure and the local pressure at sea level.

Example: Finding the current altitude using the barometer Sensor

```
finalSensorEventListenermyPressureListener = new SensorEventListener()
{
public void onSensorChanged(SensorEventsensorEvent)
{
if (sensorEvent.sensor.getType() == Sensor.TYPE_PRESSURE)
{
floatcurrentPressure = sensorEvent.values[0];
 // Calculate altitude
```

```
float altitude = SensorManager.getAltitude(
SensorManager.PRESSURE_STANDARD_ATMOSPHERE,
currentPressure);
 } }
public void onAccuracyChanged(Sensor sensor, int accuracy)
{}
};
SensorManagersm= (SensorManager)getSystemService(Context.SENSOR_SERVICE);
intsensorType = Sensor.TYPE_PRESSURE;
sm.registerListener(myPressureListener, sm.getDefaultSensor(sensorType),
SensorManager.SENSOR_DELAY_NORMAL);
```

Creating a Weather Station

To fully explore the environmental Sensors available to Android devices, the following project implements a simple weather station by monitoring the barometric pressure, ambient temperature, and ambient light levels.

1. Start by creating a new WeatherStation project that includes a WeatherStation Activity. Modify the main.xml layout resource to display three centered lines of large, bold text that will be used to display the current temperature, barometric pressure, and cloud level:

```
<?xml version=”1.0” encoding=”utf-8”?>
<LinearLayoutxmlns:android=”http://schemas.android.com/apk/res/android”
android:orientation=”vertical”
android:layout_width=”match_parent”
android:layout_height=”match_parent”>
<TextViewandroid:id=”@+id/temperature”
android:gravity=”center”
android:layout_width=”match_parent”
android:layout_height=”wrap_content”
android:textStyle=”bold”
android:textSize=”28sp”
android:text=”Temperature”
android:editable=”false”
android:singleLine=”true”
android:layout_margin=”10dp”/>
 />
```

```
<TextViewandroid:id="@+id/pressure"
android:gravity="center"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:textStyle="bold"
android:textSize="28sp"
android:text="Pressure"
android:editable="false"
android:singleLine="true"
android:layout_margin="10dp"/>
 />
<TextViewandroid:id="@+id/light"
android:gravity="center"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:textStyle="bold"
android:textSize="28sp"
android:text="Light"
android:editable="false"
android:singleLine="true"
android:layout_margin="10dp"/>
 />
</LinearLayout>
```

2. Within the WeatherStation Activity, create instance variables to store references to each of the TextView instances and the SensorManager. Also create variables to record the last recorded value obtained from each sensor:

```
privateSensorManagersensorManager;
privateTextViewtemperatureTextView;
privateTextViewpressureTextView;
privateTextViewlightTextView;
private float currentTemperature = Float.NaN;
private float currentPressure = Float.NaN;
private float currentLight = Float.NaN;
```

3. Update the onCreate method to get a reference to the three Text Views and the Sensor Manager:

```
public void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.main);
   temperatureTextView = (TextView)findViewById(R.id.temperature);
   pressureTextView = (TextView)findViewById(R.id.pressure);
   lightTextView = (TextView)findViewById(R.id.light);
sensorManager = (SensorManager)getSystemService(Context.SENSOR_SERVICE);
}
```

4. Create a new SensorEventListener implementation for each of the pressure, temperature, and light sensors. Each should simply record the last recorded value:

```
private final SensorEventListenertempSensorEventListener= new SensorEventListener() {
public void onAccuracyChanged(Sensor sensor, int accuracy) { }
public void onSensorChanged(SensorEvent event) {
currentTemperature = event.values[0];
}
};
private final SensorEventListenerpressureSensorEventListener= new SensorEventListener() {
public void onAccuracyChanged(Sensor sensor, int accuracy) { }
public void onSensorChanged(SensorEvent event) {
currentPressure = event.values[0];
}};
private final SensorEventListenerlightSensorEventListener= new SensorEventListener() {
public void onAccuracyChanged(Sensor sensor, int accuracy) { }
public void onSensorChanged(SensorEvent event) {
currentLight = event.values[0];
}};
```

USING LOCATION-BASED SERVICES

"Location-based services" is an umbrella term that describes the different technologies you can use to find a device's current location. The two main LBS elements are:

- Location Manager — Provides hooks to the location-based services.
- Location Providers — Each of these represents a different location-finding technology used to determine the device's current location.

Using the Location Manager, you can do the following:

- Obtain your current location
- Follow movement
- Set proximity alerts for detecting movement into and out of a specified area
- Find available Location Providers
- Monitor the status of the GPS receiver

Access to the location-based services is provided by the Location Manager. To access the Location Manager, request an instance of the LOCATION_SERVICE using the getSystemService method, as shown below.

```
String serviceString = Context.LOCATION_SERVICE;
LocationManagerlocationManager;
locationManager = (LocationManager)getSystemService(serviceString);
```

The following snippet shows how to request the fi ne and coarse permissions in your application manifest:

```
<uses-permissionandroid:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permissionandroid:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

USING THE EMULATOR WITH LOCATION-BASED SERVICES

Location-based services are dependent on device hardware used to find the current location. When you develop and test with the Emulator, your hardware is virtualized, and you're likely to stay in pretty much the same location.

To compensate, Android includes hooks that enable you to emulate Location Providers for testing location-based applications. In this section you learn how to mock the position of the supported GPS provider.

SELECTING A LOCATION PROVIDER

Depending on the device, you can use several technologies to determine the current location. Each technology, available as a Location Provider, offers different capabilities — including differences in power consumption, accuracy, and the ability to determine altitude, speed, or heading information.

Finding Location Providers

The LocationManager class includes static string constants that return the provider name for three Location Providers:

- LocationManager.GPS_PROVIDER
- LocationManager.NETWORK_PROVIDER
- LocationManager.PASSIVE_PROVIDER

To get a list of the names of all the providers available (based on hardware available on the device, and the permissions granted the application), call getProviders, using a Boolean to indicate if youwant all, or only the enabled, providers to be returned:

```
booleanenabledOnly = true;
List<String> providers = locationManager.getProviders(enabledOnly);
```

FINDING YOUR CURRENT LOCATION

One of the most powerful uses of location-based services is to find the physical location of the device. The accuracy of the returned location is dependent on the hardware available and the permissions requested by your application.

Location Privacy

Privacy is an important consideration when your application uses the user's location — particularly when it is regularly updating their current position. Ensure that your application uses the device location data in a way that respects the user's privacy by:

- Only using and updating location when necessary for your application
- Notifying users of when you track their locations, and if and how that location information is used, transmitted, and stored
- Allowing users to disable location updates, and respecting the system settings for LBS preferences

Finding the Last Known Location

You can find the last location fix obtained by a particular Location Provider using the getLastKnownLocation method, passing in the name of the Location Provider. The following example finds the last location fi x taken by the GPS provider:

```
String provider = LocationManager.GPS_PROVIDER;
Location location = locationManager.getLastKnownLocation(provider);
```

getLastKnownLocation does not ask the Location Provider to update the current position. If the device has not recently updated the current position, this value may not exist or be out of date. The Location object returned includes all the position information available from the provider that supplied it. This can include the time it was obtained, the accuracy of the location found, and its latitude, longitude, bearing, altitude, and speed. All these properties are available via get methods on the Location object.

Where Am I Example

The following example — Where Am I — features a new Activity that finds the device's last known location using the GPS Location Provider.

1. Create a new Where Am I project with a WhereAmI Activity. This example uses the GPS provider, so you need to include the uses-permission tag for ACCESS_FINE_LOCATION in your application manifest.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.paad.whereami"
android:versionCode="1"
android:versionName="1.0" >
<uses-sdkandroid:minSdkVersion="4" />
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"
 />
<application
android:icon="@drawable/ic_launcher"
android:label="@string/app_name" >
<activity
android:name=".WhereAmI"
android:label="@string/app_name" >
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
</application>
</manifest>
```

2. Modify the main.xml layout resource to include an android:ID attribute for the TextViewcontrol so that you can access it from within the Activity.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayoutxmlns:android="http://schemas.android.com/apk/res/android"
android:orientation="vertical"
android:layout_width="match_parent"
```

```
android:layout_height="match_parent">
<TextView
android:id="@+id/myLocationText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="@string/hello"
 />
</LinearLayout>
```

3. Override the onCreate method of the WhereAmI Activity to get a reference to the Location Manager. Call getLastKnownLocation to get the last known location, and pass it in to an updateWithNewLocation method stub.

```
@Override
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.main);
LocationManagerlocationManager;
 String svcName = Context.LOCATION_SERVICE;
locationManager = (LocationManager)getSystemService(svcName);
 String provider = LocationManager.GPS_PROVIDER;
 Location l = locationManager.getLastKnownLocation(provider);
updateWithNewLocation(l);
}
private void updateWithNewLocation(Location location) {}
```

KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM

DEPARTMENT OF INFORMATION TECHNOLOGY

20IT601PE –MOBILE APPLICATION DEVELOPMENT (KNCETR-2020)
HANDOUT

UNIT V – SMALL COMPUTING TECNOLOGY

Audio, Video Using the Camera – Using Bluetooth – Managing Network and Internet Connectivity - Managing Wi-Fi – Transferring Data Using Wi-Fi Direct – Near Field Communication.

AUDIO, VIDEO USING THE CAMERA
PLAYING AUDIO AND VIDEO

Android 4.0.3 (API level 15) supports the following multimedia formats for playback as part of the base framework. Note that some devices may support playback of additional fi le formats:

- **Audio**
  - AAC LC/LTP
  - HE-AACv1 (AAC+)
  - HE-AACv2 (Enhanced AAC+)
  - AMR-NB
  - AMR-WB
  - MP3
  - MIDI
  - Ogg Vorbis
  - PCM/WAVE
  - FLAC (on devices running Android 3.1 or above)
- **Image**
  - JPEG
  - PNG
  - WEBP (on devices running Android 4.0 or above)
  - GIF
  - BMP
- **Video**
  - H.263
  - H.264 AVC
  - MPEG-4 SP

The following network protocols are supported for streaming media:

- RTSP (RTP, SDP)
- HTTP/HTTPS progressive streaming
- HTTP/HTTPS live streaming (on devices running Android 3.0 or above)

Introducing the Media Player

The playback of audio and video within Android applications is handled primarily through the MediaPlayer class. Using the Media Player, you can play media stored in application resources, local files, Content Providers, or streamed from a network URL.

The Media Player's management of audio and video files and streams is handled as a state machine.

In the most simplistic terms, transitions through the state machine can be described as follows:

1. Initialize the Media Player with media to play.
2. Prepare the Media Player for playback.
3. Start the playback.
4. Pause or stop the playback prior to its completing.
5. The playback is complete.

To play a media resource, you need to create a new MediaPlayer instance, initialize it with a media source, and prepare it for playback.

The following section describes how to initialize and prepare the Media Player. After that, you'll learn to control the playback to start, pause, stop, or seek the prepared media.

To stream Internet media using the Media Player, your application must include the INTERNETpermission:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Android supports a limited number of simultaneous Media Player objects; not releasing them can cause runtime exceptions when the system runs out.

When you finish playback, call release on your Media Player object to free the associated resources:

```
mediaPlayer.release();
```

Preparing Audio for Playback

There are a number of ways you can play audio content through the Media Player. You can include it as an application resource, play it from local files or Content Providers, or stream it from a remote URL.

To include audio content as an application resource, add it to the res/raw folder of your resources hierarchy. Raw resources are not compressed or manipulated in any way when packaged into your application, making them an ideal way to store precompressed files such as audio files.

Initializing Audio Content for Playback

To play back audio content using a Media Player, you need to create a new Media Player object and set the data source of the audio in question. You can do this by using the static create method, passing in the Activity Context and any one of the following audio sources:

- A resource identifier (typically for an audio fi le stored in the res/raw resource folder)
- A URI to a local file (using the file:// schema)
- A URI to an online audio resource (as a URL)
- A URI to a row within a Content Provider that returns an audio file

```
// Load an audio resource from a package resource.
MediaPlayer resourcePlayer = MediaPlayer.create(this, R.raw.my_audio);
// Load an audio resource from a local file.
MediaPlayer filePlayer = MediaPlayer.create(this, Uri.parse("file:///sdcard/localfile.mp3"));
// Load an audio resource from an online resource.
MediaPlayer urlPlayer = MediaPlayer.create(this, Uri.parse("http://site.com/audio/audio.mp3"));
// Load an audio resource from a Content Provider.
MediaPlayer contentPlayer = MediaPlayer.create(this,
Settings.System.DEFAULT_RINGTONE_URI);
```

Preparing Video for Playback

Playback of video content is slightly more involved than audio. To play a video, you first must have a Surface on which to show it.

There are two alternatives for the playback of video content. The first technique, using the VideoViewclass, encapsulates the creation of a Surface and allocation and preparation of video content using a Media Player.

The second technique allows you to specify your own Surface and manipulate the underlyingMedia Player instance directly.

Creating a Surface for Video Playback

The first step to using the Media Player directly to view video content is to prepare a Surface onto which the video will be displayed.

This is generally handled using a SurfaceView object. The Surface View class is a wrapper aroundthe Surface Holder object, which, in turn, is a wrapper around the Surface that is used to supportvisual updates from background threads.

The Media Player uses a SurfaceHolder object to display video content, assigned using the setDisplay method. To include a Surface Holder in your UI layout, use the SurfaceView class, as shown in the sample layout XML.

Example 1: Sample layout using a Surface View

```
<?xml version=”1.0” encoding=”utf-8”?>
<LinearLayout
 xmlns:android=”http://schemas.android.com/apk/res/android”
 android:layout_width=”match_parent”
 android:layout_height=”match_parent”
 android:orientation=”vertical” >
<SurfaceView
 android:id=”@+id/surfaceView”
 android:layout_width=”match_parent”
 android:layout_height=”match_parent”
android:layout_weight=”30”
 />
<LinearLayout
 android:id=”@+id/linearLayout1”
 android:layout_width=”match_parent”
 android:layout_height=”wrap_content”
android:layout_weight=”1”>
<Button
 android:id=”@+id/buttonPlay”
 android:layout_width=”wrap_content”
 android:layout_height=”wrap_content”
android:text=”Play”
 />
<Button
 android:id=”@+id/buttonPause”
 android:layout_width=”wrap_content”
 android:layout_height=”wrap_content”
 android:text=”Pause”
 />
<Button
 android:id=”@+id/buttonSkip”
 android:layout_width=”wrap_content”
 android:layout_height=”wrap_content”
```

```
 android:text="Skip"
 />
</LinearLayout>
</LinearLayout>
```

Surface Holders are created asynchronously, so you must wait until the surfaceCreated handlerhas been fi red before assigning the returned Surface Holder object to the Media Player by implementing the SurfaceHolder.Callback interface.

After creating and assigning the Surface Holder to your Media Player, use the setDataSourcemethod to specify the path, URL, or Content Provider URI of the video resource to play.

Example 2: Initializing and assigning a Surface View to a Media Player

```
import java.io.IOException;
import android.app.Activity;
import android.media.MediaPlayer;
import android.os.Bundle;
import android.util.Log;
import android.view.SurfaceHolder;
import android.view.SurfaceView;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
public class SurfaceViewVideoViewActivity extends Activity
implements SurfaceHolder.Callback {
static final String TAG = "SurfaceViewVideoViewActivity";
private MediaPlayer mediaPlayer;
public void surfaceCreated(SurfaceHolder holder) {
try {
// When the surface is created, assign it as the
 // display surface and assign and prepare a data
 // source.
mediaPlayer.setDisplay(holder);
mediaPlayer.setDataSource("/sdcard/test2.3gp");
mediaPlayer.prepare();
 } catch (IllegalArgumentException e) {
```

```
Log.e(TAG, “Illegal Argument Exception”, e);
 } catch (IllegalStateException e) {
Log.e(TAG, “Illegal State Exception“, e);
 } catch (SecurityException e) {
Log.e(TAG, “Security Exception“, e);
 } catch (IOException e) {
Log.e(TAG, “IO Exception“, e);
 } }
public void surfaceDestroyed(SurfaceHolder holder) {
mediaPlayer.release(); }
public void surfaceChanged(SurfaceHolder holder,int format, int width, int height)
{ }
 @Override
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.surfaceviewvideoviewer);
 // Create a new Media Player.
mediaPlayer = new MediaPlayer();
 // Get a reference to the Surface View.
final SurfaceView surfaceView =
 (SurfaceView)findViewById(R.id.surfaceView);
 // Configure the Surface View.
surfaceView.setKeepScreenOn(true);
 // Configure the Surface Holder and register the callback.
 SurfaceHolder holder = surfaceView.getHolder();
holder.addCallback(this);
holder.setType(SurfaceHolder.SURFACE_TYPE_PUSH_BUFFERS);
holder.setFixedSize(400, 300);
// Connect a play button.
 Button playButton = (Button)findViewById(R.id.buttonPlay);
playButton.setOnClickListener(new OnClickListener() {
public void onClick(View v) {
mediaPlayer.start();}
});
```

```
// Connect a pause button.
Button pauseButton = (Button)findViewById(R.id.buttonPause);
pauseButton.setOnClickListener(new OnClickListener() {
public void onClick(View v) {
mediaPlayer.pause();
} });
// Add a skip button.
Button skipButton = (Button)findViewById(R.id.buttonSkip);
skipButton.setOnClickListener(new OnClickListener() {
public void onClick(View v) {
mediaPlayer.seekTo(mediaPlayer.getDuration()/2);
}
});
}}
```

Controlling Media Player Playback

When a Media Player is prepared, call start to begin playback of the associated media:

```
mediaPlayer.start();
```

Use the stop and pause methods to stop or pause playback, respectively.

The Media Player also provides the getDuration method to find the length of the media beingplayed and the getCurrentPosition method to find the playback position. Use the seekTo methodto jump to a specific position in the media (refer to Example2).

To ensure a consistent media control experience, Android includes the MediaController — a standard control that provides the common media control buttons, as shown in Figure 1.

FIGURE 1

If you're using a Video View to display your video content, you can use the Media Controller simply by the Video View's setMediaController method:

```
// Attach a Media Controller
MediaController mediaController = new MediaController(this);
videoView.setMediaController(mediaController);
```

BLUETOOTH, NFC, NETWORKS, AND WI-FI

BLUETOOTH

Using Bluetooth

Bluetooth is a communications protocol designed for short-range, low-bandwidth peer-to-peer communications.

Using the Bluetooth APIs, you can search for, and connect to, other Bluetooth devices within range. By initiating a communications link using Bluetooth Sockets, you can then transmit and receive streams of data between devices from within your applications.

Managing the Local Bluetooth Device Adapter

The local Bluetooth device is controlled via the BluetoothAdapter class, which represents the host Android device on which your application is running.

To access the default Bluetooth Adapter, call getDefaultAdapter, as shown below. SomeAndroid devices feature multiple Bluetooth adapters, though it is currently only possible to access the default device.

For accessing the default Bluetooth Adapter:

```
BluetoothAdapter bluetooth = BluetoothAdapter.getDefaultAdapter();
```

To read any of the local Bluetooth Adapter properties, initiate discovery, or fi nd bonded devices, you need to include the BLUETOOTH permission in your application manifest. To modify any of the local device properties, the BLUETOOTH_ADMIN permission is also required:

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
```

The Bluetooth Adapter offers methods for reading and setting properties of the local Bluetooth hardware.

Use the isEnabled method to confirm the device is enabled, after which you can access the Bluetooth Adapter's friendly name (an arbitrary string that users can set to identify a particular device) and hardware address, using the getName and getAddress methods, respectively:

```
if (bluetooth.isEnabled()) {
 String address = bluetooth.getAddress();
 String name = bluetooth.getName();
}
```

If you have the BLUETOOTH_ADMIN permission, you can change the friendly name of the Bluetooth Adapter using the setName method:

bluetooth.setName("Blackfang");

To find a more detailed description of the current Bluetooth Adapter state, use the getStatemethod, which will return one of the following BluetoothAdapter constants:

- STATE_TURNING_ON
- STATE_ON
- STATE_TURNING_OFF
- STATE_OFF

To conserve battery life and optimize security, most users will keep Bluetooth disabled until they plan to use it.

To enable the Bluetooth Adapter, you can start a system Preference Activity using the BluetoothAdapter.ACTION_REQUEST_ENABLE static constant as a startActivityForResult action string:

```
startActivityForResult(new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE), 0);
```

Example: Enabling Bluetooth

```
private static final int ENABLE_BLUETOOTH = 1;
private void initBluetooth() {
if (!bluetooth.isEnabled()) {
 // Bluetooth isn't enabled, prompt the user to turn it on.
 Intent intent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
startActivityForResult(intent, ENABLE_BLUETOOTH);
 } else {
 // Bluetooth is enabled, initialize the UI.
initBluetoothUI(); }}
protected void onActivityResult(int requestCode,
int resultCode, Intent data) {
if (requestCode == ENABLE_BLUETOOTH)
if (resultCode == RESULT_OK) {
 // Bluetooth has been enabled, initialize the UI.
initBluetoothUI();
 }}
```

Bluetooth Communications

The Android Bluetooth communications APIs are wrappers around RFCOMM, the Bluetooth radio frequency communications protocol. RFCOMM supports RS232 serial communication over the Logical Link Control and Adaptation Protocol (L2CAP) layer.

In practice, this alphabet soup provides a mechanism for opening communication sockets between two paired Bluetooth devices.

You can establish an RFCOMM communication channel for bidirectional communications using the following classes.

- BluetoothServerSocket — Used to establish a listening socket for initiating a link between devices. To establish a handshake, one device acts as a server to listen for, and accept, incoming connection requests.
- BluetoothSocket — Used to create a new client to connect to a listening Bluetooth Server Socket. Also returned by the Bluetooth Server Socket after a connection is established. Once a connection is established, Bluetooth Sockets are used by both the server and client to transfer data streams.

Example: Listening for Bluetooth Socket connection requests

```
private BluetoothSocket transferSocket;
private UUID startServerSocket(BluetoothAdapter bluetooth) {
 UUID uuid = UUID.fromString("a60f35f0-b93a-11de-8a39-08002009c666");
 String name = "bluetoothserver";
try {
final BluetoothServerSocket btserver =
bluetooth.listenUsingRfcommWithServiceRecord(name, uuid);
 Thread acceptThread = new Thread(new Runnable() {
public void run() {
try {
 // Block until client connection established.
 BluetoothSocket serverSocket = btserver.accept();
 // Start listening for messages.
listenForMessages(serverSocket);
 // Add a reference to the socket used to send messages.
transferSocket = serverSocket;
 } catch (IOException e) {
```

```
Log.e("BLUETOOTH", "Server connection IO Exception", e);
 } } });
acceptThread.start();
 } catch (IOException e) {
Log.e("BLUETOOTH", "Socket listener IO Exception", e);
 }
return uuid;}
```

Selecting Remote Bluetooth Devices for Communications

The BluetoothSocket class can be used on the client device to initiate a communications channel from within your application to a listening Bluetooth Server Socket. It is also returned by the Bluetooth Server Socket Listener after a connection to a client device has been established.

Create a client-side Bluetooth Socket by calling create RfcommSocketToServiceRecord on a BluetoothDevice object that represents the target remote server device. The target device should have a Bluetooth Server Socket listening for connection requests (as described in the previous section).

There are a number of ways to obtain a reference to a remote Bluetooth Device, and some important caveats regarding the devices with which you can create a communications link.

Bluetooth Device Connection Requirements

In order for a Bluetooth Socket to establish a connection to a remote Bluetooth Device, the following conditions must be true:

- The remote device must be discoverable.
- The remote device must be accepting connections through a Bluetooth Server Socket.
- The local and remote devices must be paired (bonded). If the devices are not paired, the users of each device will be prompted to pair them when the connection request is initiated.

Example: Creating a Bluetooth client socket

```
private void connectToServerSocket(BluetoothDevice device, UUID uuid) {
try{
 BluetoothSocket clientSocket = device.createRfcommSocketToServiceRecord(uuid);
 // Block until server connection accepted.
clientSocket.connect();
```

```
 // Start listening for messages.
listenForMessages(clientSocket);
 // Add a reference to the socket used to send messages.
transferSocket = clientSocket;
 } catch (IOException e) {
Log.e("BLUETOOTH", "Bluetooth client I/O Exception", e);
}}
```

MANAGING NETWORK AND INTERNET CONNECTIVITY

With the speed, reliability, and cost of Internet connectivity being dependent on the network technology used (Wi-Fi, GPRS, 3G, LTE, and so on), letting your applications know and manage these connections can help to ensure they run efficiently and responsively.

Android broadcasts Intents that allow you to monitor changes in network connectivity and offers APIs that provide control over network settings and connections.

Android networking is principally handled via the ConnectivityManager, a Service that lets you monitor the connectivity state, set your preferred network connection, and manage connectivity failover.

The section "Managing Wi-Fi" describes how to use the WifiManager to monitor and control the device's Wi-Fi connectivity specifically. The WifiManager lets you create new Wi-Fi configurations, monitor and modify the existing Wi-Fi network settings, manage the active connection, and perform access point scans.

Introducing the Connectivity Manager

The ConnectivityManager represents the Network Connectivity Service. It's used to monitor the state of network connections, configure failover settings, and control the network radios.

To use the Connectivity Manager, your application needs read and write network state access permissions:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.CHANGE_NETWORK_STATE"/>
```

To access the Connectivity Manager, use getSystemService, passing in Context.CONNECTIVITY_SERVICE as the service name, as shown below:

```
String service = Context.CONNECTIVITY_SERVICE;
ConnectivityManager connectivity = (ConnectivityManager)getSystemService(service);
```

Supporting User Preferences for Background Data Transfers

Until Android 4.0 (API level 14), user preferences for background data transfers were enforced at the application level — meaning that for pre-Android 4.0 platforms, you are responsible for adhering to the user's preference for allowing background data transfers.

To obtain the background data setting, call the getBackgroundDataSetting method on the Connectivity Manager object:

```
boolean backgroundEnabled = connectivity.getBackgroundDataSetting();
```

If the user does change the background data preference, the system will send a Broadcast Intent with the Connectivity Manager's ACTION_BACKGROUND_DATA_SETTING_CHANGED action.

To monitor changes in the background data setting, create and register a new Broadcast Receiver that listens for this Broadcast Intent, as listed below.

```
registerReceiver(
new BroadcastReceiver() {
 @Override
public void onReceive(Context context, Intent intent) {
boolean backgroundEnabled =
connectivity.getBackgroundDataSetting();
setBackgroundData(backgroundEnabled);
 } },
new IntentFilter(
 ConnectivityManager.ACTION_BACKGROUND_DATA_SETTING_CHANGED)
);
```

MANAGING WI-FI

The WifiManager, which represents the Android Wi-Fi Connectivity Service, can be used to configure Wi-Fi network connections, manage the current Wi-Fi connection, scan for access points, and monitor changes in Wi-Fi connectivity.

To use the Wi-Fi Manager, your application must have uses-permissions for accessing and changing the Wi-Fi state included in its manifest:

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
```

Access the Wi-Fi Manager using the getSystemService method, passing in the Context.WIFI_SERVICE constant, as shown below:

```
String service = Context.WIFI_SERVICE;
WifiManager wifi = (WifiManager)getSystemService(service);
```

To Monitoring and changing Wi-Fi state:

```
if (!wifi.isWifiEnabled())
if (wifi.getWifiState() != WifiManager.WIFI_STATE_ENABLING)
wifi.setWifiEnabled(true);
```

Monitoring Wi-Fi Connectivity

In most cases its best practice to use the Connectivity Manager to monitor changes in Wi-Fi connectivity; however, the Wifi Manager does broadcast Intents whenever the connectivity status of the Wi-Fi network changes, using an action from one of the following constants defined in theWifiManager class:

- WIFI_STATE_CHANGED_ACTION — Indicates that the Wi-Fi hardware status has changed, moving between enabling, enabled, disabling, disabled, and unknown. It includes two extra values keyed on EXTRA_WIFI_STATE and EXTRA_PREVIOUS_STATE that provide the new and previous Wi-Fi states, respectively.
- SUPPLICANT_CONNECTION_CHANGE_ACTION — This Intent is broadcast whenever the connection state with the active supplicant (access point) changes.
- NETWORK_STATE_CHANGED_ACTION — Fired whenever the Wi-Fi connectivity state changes. This Intent includes two extras: the first, EXTRA_NETWORK_INFO, includes a NetworkInfo object that details the current network state, whereas the second, EXTRA_BSSID, includes the BSSID of the access point you're connected to.
- RSSI_CHANGED_ACTION — You can monitor the current signal strength of the connected Wi-Fi network by listening for the RSSI_CHANGED_ACTION Intent. This Broadcast Intent includes an integer extra, EXTRA_NEW_RSSI, that holds the current signal strength.

Monitoring Active Wi-Fi Connection Details

When an active Wi-Fi connection has been established, you can use the getConnectionInfomethod on the Wi-Fi Manager to find information on the connection's status. The returned WifiInfo object includes the SSID, BSSID, MAC address, and IP address of the current accesspoint, as well as the current link speed and signal strength, as shown below:

```
WifiInfo info = wifi.getConnectionInfo();
if (info.getBSSID() != null) {
```

```
int strength = WifiManager.calculateSignalLevel(info.getRssi(), 5);
int speed = info.getLinkSpeed();
 String units = WifiInfo.LINK_SPEED_UNITS;
 String ssid = info.getSSID();
 String cSummary =String.format("Connected to %s at %s%s.
 Strength %s/5",
ssid, speed, units, strength);
Log.d(TAG, cSummary);
}
```

TRANSFERRING DATA USING WI-FI DIRECT

Wi-Fi Direct is a communications protocol designed for medium-range, high-bandwidth peer-to-peer communications. Support for Wi-Fi Direct was added to Android 4.0 (API level 14). Compared to Bluetooth, Wi-Fi Direct is faster and more reliable, and works over greater distances.

Using the Wi-Fi Direct APIs, you can search for, and connect to, other Wi-Fi Direct devices within range. By initiating a communications link using sockets, you can then transmit and receive streams of data between supported devices (including some printers, scanners, cameras, and televisions) and between instances of your application running on different devices.

Initializing the Wi-Fi Direct Framework

To use Wi-Fi Direct, your application requires the ACCESS_WIFI_STATE, CHANGE_WIFI_STATE, and INTERNET permissions:

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.INTERNET"/>
```

Wi-Fi Direct connections are initiated and managed using the WifiP2pManager system service, which you canaccess using the getSystemService method, passing in the Context.WIFI_P2P_SERVICE constant:

```
wifiP2pManager = (WifiP2pManager)getSystemService(Context.WIFI_P2P_SERVICE);
```

Before you can use the WiFi P2P Manager, you must create a channel to the Wi-Fi Direct framework using the Wifi P2P Manager's initialize method. Pass in the current Context, the Looper on which to receive Wi-Fi Direct events, and a ChannelListener to listen for the loss of your channel connection, as shown below:

```
private WifiP2pManager wifiP2pManager;
private Channel wifiDirectChannel;
```

```
private void initializeWiFiDirect() {
 wifiP2pManager =
 (WifiP2pManager)getSystemService(Context.WIFI_P2P_SERVICE);
wifiDirectChannel = wifiP2pManager.initialize(this, getMainLooper(),
new ChannelListener() {
public void onChannelDisconnected() {
initializeWiFiDirect();
 } } );}
```

NEAR FIELD COMMUNICATION

Android 2.3 (API level 9) introduced Near Field Communication (NFC) APIs. NFC is a contactless technology used to transmit small amounts of data across short distances (typically less than 4 centimeters).

NFC transfers can occur between two NFC-enabled devices, or between a device and an NFC "tag." Tags can range from passive tags that transmit a URL when scanned to complex systems such as those used in NFC payment solutions, such as Google Wallet.

NFC messages in Android are handled using the NFC Data Exchange Format (NDEF). In order to read, write, or broadcast NFC messages, your application requires the NFC manifest permission:

```
<uses-permission android:name="android.permission.NFC" />
```

Reading NFC Tags

When an Android device is used to scan an NFC tag, the system will decode the incoming payload using its own tag dispatch system, which analyzes the tag, categorizes the data, and uses an Intent tolaunch an application to receive the data.

For an application to receive NFC data, you need to add an Activity Intent Filter that listens for one of the following Intent actions:

- NfcAdapter.ACTION_NDEF_DISCOVERED — The highest priority, and most specific, of the NFC messages. Intents using this action include MIME types and/or URI data. Its best practice to listen for this broadcast whenever possible because the extra data allows you to be more specific in defining which tags to respond to.
- NfcAdapter.ACTION_TECH_DISCOVERED — This action is broadcast when the NFC technology is known but the tag contains no data — or contains data that can't be mapped to a MIME type or URI.
- NfcAdapter.ACTION_TAG_DISCOVERED — If a tag is received from an unknown technology, it will be broadcast using this action.

Example: Listening for NFC tags

```
<activity android:name=".BlogViewer">
<intent-filter>
<action android:name="android.nfc.action.NDEF_DISCOVERED"/>
<category android:name="android.intent.category.DEFAULT"/>
<data android:scheme="http"
 android:host="blog.radioactiveyak.com"/>
</intent-filter>
</activity>
```

Its good practice to make your NFC Intent Filters as specific as possible to minimize the number of applications available to respond to a given NFC tag and provide the best, fastest user experience.

The NfcAdapter.EXTRA_TAG extra includes a raw Tag object that represents the scanned tag. The NfcAdapter.EXTRA_TNDEF_MESSAGES extra contains an array of NDEF Messages, as shown in below:

```
String action = getIntent().getAction();
if (NfcAdapter.ACTION_NDEF_DISCOVERED.equals(action)) {
Parcelable[] messages =
 intent.getParcelableArrayExtra(NfcAdapter.EXTRA_NDEF_MESSAGES);
for (int i = 0; i < messages.length; i++) {
 NdefMessage message = (NdefMessage)messages[i];
NdefRecord[] records = message.getRecords();
for (int j = 0; j < records.length; j++) {
 NdefRecord record = records[j];
 // TODO Process the individual records.
 } }
}
```